```
cloudra/__init__.py:1   """Cloudra — AI Agent collaboration framework for chemical research."""
cloudra/__init__.py:2
cloudra/__init__.py:3   from importlib import import_module
cloudra/__init__.py:4
cloudra/__init__.py:5   __version__ = "0.1.0rc1"
cloudra/__init__.py:6
cloudra/__init__.py:7   # 兼容垫片：包结构重排（core/services/persistence/base/infra）前的
cloudra/__init__.py:8   # `from cloudra import X` 老路径全部继续可用（惰性加载，不提前拉起重依赖）。
cloudra/__init__.py:9   # 执行栈（cloudra.core.agent / message_history）已删除，不再留入口。
cloudra/__init__.py:10  _LAZY_MODULES = {
cloudra/__init__.py:11      # services
cloudra/__init__.py:12      "session_store": "cloudra.services.session_store",
cloudra/__init__.py:13      "nodes": "cloudra.services.nodes",
cloudra/__init__.py:14      # persistence
cloudra/__init__.py:15      "database": "cloudra.persistence.database",
cloudra/__init__.py:16      "artifacts": "cloudra.persistence.artifacts",
cloudra/__init__.py:17      "memory": "cloudra.persistence.memory",
cloudra/__init__.py:18      "paths": "cloudra.persistence.paths",
cloudra/__init__.py:19      "fs_utils": "cloudra.persistence.fs_utils",
cloudra/__init__.py:20      "workspace": "cloudra.persistence.workspace",
cloudra/__init__.py:21      # base
cloudra/__init__.py:22      "session_class": "cloudra.base.session_class",
cloudra/__init__.py:23      "skill_class": "cloudra.base.skill_class",
cloudra/__init__.py:24      "tools_class": "cloudra.base.tools_class",
cloudra/__init__.py:25      # infra
cloudra/__init__.py:26      "config": "cloudra.infra.config",
cloudra/__init__.py:27      "llm_client": "cloudra.infra.llm_client",
cloudra/__init__.py:28      "streaming": "cloudra.infra.streaming",
cloudra/__init__.py:29      "token_estimator": "cloudra.infra.token_estimator",
cloudra/__init__.py:30      "web_search_native": "cloudra.infra.web_search_native",
cloudra/__init__.py:31      "slurm": "cloudra.infra.slurm",
cloudra/__init__.py:32      "compute_utils": "cloudra.infra.compute_utils",
cloudra/__init__.py:33      "operations": "cloudra.infra.operations",
cloudra/__init__.py:34      "yolo_guard": "cloudra.infra.yolo_guard",
cloudra/__init__.py:35      "platform_utils": "cloudra.infra.platform_utils",
cloudra/__init__.py:36      "runtime_profile": "cloudra.infra.runtime_profile",
cloudra/__init__.py:37  }
cloudra/__init__.py:38
cloudra/__init__.py:39  __all__ = [*_LAZY_MODULES, "__version__"]
cloudra/__init__.py:40
cloudra/__init__.py:41
cloudra/__init__.py:42  def __getattr__(name: str):
cloudra/__init__.py:43      """Keep legacy package module paths without importing heavy deps eagerly."""
cloudra/__init__.py:44      module_path = _LAZY_MODULES.get(name)
cloudra/__init__.py:45      if module_path is not None:
cloudra/__init__.py:46          value = import_module(module_path)
cloudra/__init__.py:47          globals()[name] = value
cloudra/__init__.py:48          return value
cloudra/__init__.py:49      raise AttributeError(f"module 'cloudra' has no attribute {name!r}")
cloudra/__init__.py:50
cloudra/__init__.py:51
cloudra/__init__.py:52  def __dir__() -> list[str]:
```

```
cloudra/__init__.py:53      return sorted(set(globals()) | set(__all__))
cloudra/__main__.py:1   """Allow running as `python -m cloudra`."""
cloudra/__main__.py:2   from cloudra.cli import main
cloudra/__main__.py:3
cloudra/__main__.py:4   if __name__ == "__main__":
cloudra/__main__.py:5       main()
cloudra/base/__init__.py:1  """cloudra.base 子包。"""
cloudra/base/role_identity.py:1 """DAG 存储词表与 v2 role_id 之间的唯一映射。
cloudra/base/role_identity.py:2
cloudra/base/role_identity.py:3 DAG 变更日志、CallReview sidecar 与 process-node ``assignee`` 用的是历史 a-N
cloudra/base/role_identity.py:4 词表（``a-0`` = Coordinator，``a-1``..``a-8`` = 八个科研子 Agent，
cloudra/base/role_identity.py:5 ``a-supervisor`` = 科学督导）；runtime_v2 的实时身份是语义 role_id
cloudra/base/role_identity.py:6 （``coordinator`` / ``small_mol`` …）。两套词表只在**存储/读取边界**互换，
cloudra/base/role_identity.py:7 映射表只此一张——之前 subagent_bridge 的 ``_AGENT_ID_TO_ROLE``、
cloudra/base/role_identity.py:8 ``subagents/registry.py`` 的 a-N 列表与 launcher/swarm 提示词里的角色清单各写一份，
cloudra/base/role_identity.py:9 新增角色要同步多处。
cloudra/base/role_identity.py:10
cloudra/base/role_identity.py:11    放置理由：runtime_v2 侧（RoleProfile）与工具侧、DAG 侧都要读它，
cloudra/base/role_identity.py:12    所以放在最底层的 ``cloudra.base``，不引入反向依赖。
cloudra/base/role_identity.py:13    """
cloudra/base/role_identity.py:14
cloudra/base/role_identity.py:15    from __future__ import annotations
cloudra/base/role_identity.py:16
cloudra/base/role_identity.py:17    #: role_id → DAG 存储词表。新增角色只需在此登记一次 + 在 BUILTIN_ROLES 声明
cloudra/base/role_identity.py:18    #: （roles.py 里的 RoleProfile.legacy_agent_id 直接取自本表，不手抄）。
cloudra/base/role_identity.py:19    LEGACY_AGENT_ID_BY_ROLE: dict[str, str] = {
cloudra/base/role_identity.py:20        "coordinator": "a-0",
cloudra/base/role_identity.py:21        "explorer": "a-1",
cloudra/base/role_identity.py:22        "writer": "a-2",
cloudra/base/role_identity.py:23        "researcher": "a-3",
cloudra/base/role_identity.py:24        "small_mol": "a-4",
cloudra/base/role_identity.py:25        "md": "a-5",
cloudra/base/role_identity.py:26        "dft": "a-6",
cloudra/base/role_identity.py:27        "experiment": "a-7",
cloudra/base/role_identity.py:28        "env": "a-8",
cloudra/base/role_identity.py:29        "supervisor": "a-supervisor",
cloudra/base/role_identity.py:30    }
cloudra/base/role_identity.py:31
cloudra/base/role_identity.py:32    #: 反向索引（一次构建）。
cloudra/base/role_identity.py:33    _ROLE_BY_LEGACY_AGENT_ID: dict[str, str] = {
cloudra/base/role_identity.py:34        legacy: role for role, legacy in LEGACY_AGENT_ID_BY_ROLE.items()
cloudra/base/role_identity.py:35    }
cloudra/base/role_identity.py:36
cloudra/base/role_identity.py:37
cloudra/base/role_identity.py:38    def legacy_agent_id_for_role(role_id: str) -> str:
cloudra/base/role_identity.py:39        """v2 role_id → DAG 词表 id。未知 role（或传入的已是 legacy id）原样透传。
cloudra/base/role_identity.py:40
cloudra/base/role_identity.py:41        透传是刻意的：历史库里存着 a-N，读取边界不能再把未知值改成别的身份。
cloudra/base/role_identity.py:42        """
cloudra/base/role_identity.py:43        return LEGACY_AGENT_ID_BY_ROLE.get(role_id, role_id)
cloudra/base/role_identity.py:44
cloudra/base/role_identity.py:45
```

```
cloudra/base/role_identity.py:46    def role_id_for_legacy_agent_id(legacy_agent_id: str) -> str:
cloudra/base/role_identity.py:47        """DAG 词表 id → v2 role_id。未知值（或已是 role_id）原样透传。
cloudra/base/role_identity.py:48
cloudra/base/role_identity.py:49        仅用于把历史输入（旧提示词/旧工具参数里的 ``a-3``）归一到语义身份；新事件与
cloudra/base/role_identity.py:50        新工具参数一律以 role_id 为准。
cloudra/base/role_identity.py:51        """
cloudra/base/role_identity.py:52        return _ROLE_BY_LEGACY_AGENT_ID.get(legacy_agent_id, legacy_agent_id)
cloudra/base/session_class.py:1
cloudra/base/session_class.py:2 import os
cloudra/base/session_class.py:3 from datetime import datetime
cloudra/base/session_class.py:4
cloudra/base/session_class.py:5 from cloudra.persistence.fs_utils import atomic_write_json
cloudra/base/session_class.py:6
cloudra/base/session_class.py:7
cloudra/base/session_class.py:8 class Session:
cloudra/base/session_class.py:9     """内存态会话镜像；数据库（session 表）是唯一持久化事实来源。
cloudra/base/session_class.py:10
cloudra/base/session_class.py:11        默认不再写 per-session JSON 快照文件；调试时可设置环境变量
cloudra/base/session_class.py:12        ``CLOUDRA_DEBUG_SESSION_JSON=1`` 恢复写文件行为（upload_json 仍可显式调用）。
cloudra/base/session_class.py:13        """
cloudra/base/session_class.py:14        id: str
cloudra/base/session_class.py:15        path: str
cloudra/base/session_class.py:16        create_time: datetime
cloudra/base/session_class.py:17        state: str = 'sleeping'
cloudra/base/session_class.py:18
cloudra/base/session_class.py:19        def __init__(self, id: str, path: str) -> None:
cloudra/base/session_class.py:20            self.create_time = datetime.now()
cloudra/base/session_class.py:21            self.id = id
cloudra/base/session_class.py:22            self.path = path
cloudra/base/session_class.py:23            self.state = 'sleeping'
cloudra/base/session_class.py:24            if os.environ.get("CLOUDRA_DEBUG_SESSION_JSON") == "1":
cloudra/base/session_class.py:25                dir_name = os.path.dirname(self.path)
cloudra/base/session_class.py:26                if dir_name:
cloudra/base/session_class.py:27                    os.makedirs(dir_name, exist_ok=True)
cloudra/base/session_class.py:28                self.upload_json()
cloudra/base/session_class.py:29
cloudra/base/session_class.py:30
cloudra/base/session_class.py:31        def to_json(self) -> dict:
cloudra/base/session_class.py:32            create_time = self.create_time
cloudra/base/session_class.py:33            if isinstance(create_time, datetime):
cloudra/base/session_class.py:34                create_time = create_time.isoformat()
cloudra/base/session_class.py:35            json_dict = {
cloudra/base/session_class.py:36                'id': self.id,
cloudra/base/session_class.py:37                'path': self.path,
cloudra/base/session_class.py:38                'create_time': create_time,
cloudra/base/session_class.py:39                'state': self.state,
cloudra/base/session_class.py:40            }
cloudra/base/session_class.py:41            return json_dict
cloudra/base/session_class.py:42
cloudra/base/session_class.py:43        def upload_json(self):
cloudra/base/session_class.py:44            atomic_write_json(self.path, self.to_json())
cloudra/base/session_class.py:45
```

```
cloudra/base/skill_class.py:1   import hashlib
cloudra/base/skill_class.py:2   import os
cloudra/base/skill_class.py:3
cloudra/base/skill_class.py:4   import frontmatter as frt
cloudra/base/skill_class.py:5
cloudra/base/skill_class.py:6   # 模块级缓存按 SKILL.md 内容 revision 失效。Phase 6 后 skill manifest
cloudra/base/skill_class.py:7   # 允许在进程运行期间更新，不能再假设目录在整个进程生命周期中静态不变。
cloudra/base/skill_class.py:8   _SKILL_DATAS_CACHE: dict[str, tuple[str, list[dict]]] = {}
cloudra/base/skill_class.py:9
cloudra/base/skill_class.py:10  # NOTE: Skills 模型仅作为 skill frontmatter 的字段参照，当前未被实例化使用。
cloudra/base/skill_class.py:11  # 若后续需要校验 skill 元数据，可取消注释并补全默认值。
cloudra/base/skill_class.py:12  # class Skills(BaseModel):
cloudra/base/skill_class.py:13  #     name: str
cloudra/base/skill_class.py:14  #     description: str
cloudra/base/skill_class.py:15  #     path: str
cloudra/base/skill_class.py:16  #     scripts: list = []
cloudra/base/skill_class.py:17  #     references: list = []
cloudra/base/skill_class.py:18
cloudra/base/skill_class.py:19
cloudra/base/skill_class.py:20  class skill_loader:
cloudra/base/skill_class.py:21
cloudra/base/skill_class.py:22      # Skills are bundled with the package, not user data（base/ 上溯一级到包根）
cloudra/base/skill_class.py:23      base_path: str = os.path.join(
cloudra/base/skill_class.py:24          os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'skills'
cloudra/base/skill_class.py:25      )
cloudra/base/skill_class.py:26
cloudra/base/skill_class.py:27      def __init__(self):
cloudra/base/skill_class.py:28          # FIX: 原代码直接用了模块级变量 base_path（缺少 self.），在实例方法中访问类属性应加 self
cloudra/base/skill_class.py:29          self.namelst = os.listdir(self.base_path)
cloudra/base/skill_class.py:30
cloudra/base/skill_class.py:31          self.pathlst = [os.path.join(self.base_path, skill_name) for skill_name in self.
namelst]
cloudra/base/skill_class.py:32
cloudra/base/skill_class.py:33          self.md_pathlst = [os.path.join(path,'SKILL.md') for path in self.pathlst if os.
path.isdir(path)]
cloudra/base/skill_class.py:34
cloudra/base/skill_class.py:35      def skill_datas_get(self) -> list[dict]:
cloudra/base/skill_class.py:36          revision_hash = hashlib.sha256()
cloudra/base/skill_class.py:37          for md in sorted(self.md_pathlst):
cloudra/base/skill_class.py:38              revision_hash.update(md.encode("utf-8"))
cloudra/base/skill_class.py:39              try:
cloudra/base/skill_class.py:40                  with open(md, "rb") as stream:
cloudra/base/skill_class.py:41                      revision_hash.update(stream.read())
cloudra/base/skill_class.py:42              except OSError:
cloudra/base/skill_class.py:43                  continue
cloudra/base/skill_class.py:44          revision = revision_hash.hexdigest()
cloudra/base/skill_class.py:45          cached = _SKILL_DATAS_CACHE.get(self.base_path)
cloudra/base/skill_class.py:46          if cached is None or cached[0] != revision:
cloudra/base/skill_class.py:47              mds = self.md_pathlst
cloudra/base/skill_class.py:48              datas = []
cloudra/base/skill_class.py:49              for md in mds:
cloudra/base/skill_class.py:50                  with open(md, 'r', encoding='utf-8') as f:
cloudra/base/skill_class.py:51                      post = frt.load(f)
cloudra/base/skill_class.py:52                      metadata = post.metadata
```

```
cloudra/base/skill_class.py:53                    content = post.content
cloudra/base/skill_class.py:54
cloudra/base/skill_class.py:55                    # FIX: dict.update() 返回 None，原代码把 None 当数据 append 了
cloudra/base/skill_class.py:56                    # 这里直接修改 metadata dict 后 append（frontmatter 的 metadata 是普通 dict
）
cloudra/base/skill_class.py:57                    metadata['content'] = content
cloudra/base/skill_class.py:58                    datas.append(metadata)
cloudra/base/skill_class.py:59
cloudra/base/skill_class.py:60                _SKILL_DATAS_CACHE[self.base_path] = (revision, datas)
cloudra/base/skill_class.py:61
cloudra/base/skill_class.py:62            return _SKILL_DATAS_CACHE[self.base_path][1]
cloudra/base/skill_class.py:63
cloudra/base/skill_class.py:64        def get_reference(self, skill_name, ref_name) -> str:
cloudra/base/skill_class.py:65            # 支持多种可能的资料文件夹命名约定
cloudra/base/skill_class.py:66            possible_dirs = ['references', 'reference', 'ref']
cloudra/base/skill_class.py:67
cloudra/base/skill_class.py:68            for dir_name in possible_dirs:
cloudra/base/skill_class.py:69                ref_path = os.path.join(self.base_path, skill_name, dir_name, ref_name)
cloudra/base/skill_class.py:70                if os.path.isfile(ref_path):
cloudra/base/skill_class.py:71                    with open(ref_path, 'r', encoding='utf-8') as f:
cloudra/base/skill_class.py:72                        return f.read()
cloudra/base/skill_class.py:73                elif os.path.isdir(ref_path):
cloudra/base/skill_class.py:74                    dir_filelst = os.listdir(ref_path)
cloudra/base/skill_class.py:75                    return fr'该reference为文件夹，文件夹下有：{dir_filelst} ,请根据需要输入 {ref_name
}/filename 调用资料，win系统请用 {ref_name}\filename'
cloudra/base/skill_class.py:76
cloudra/base/skill_class.py:77            return '该资料不存在。'
cloudra/base/skill_class.py:78
cloudra/base/skill_class.py:79        # FIX: 原拼写为 skill_dsatas_get（多打了个 s）
cloudra/base/skill_class.py:80        def get_content(self, skill_name) -> str:
cloudra/base/skill_class.py:81            datas = self.skill_datas_get()
cloudra/base/skill_class.py:82            for data in datas:
cloudra/base/skill_class.py:83                if data['name'] == skill_name:
cloudra/base/skill_class.py:84                    return data['content']
cloudra/base/skill_class.py:85            return 'skill不存在。'
cloudra/base/skill_class.py:86
cloudra/base/skill_class.py:87
cloudra/base/skill_class.py:88
cloudra/base/skill_class.py:89
cloudra/base/skill_class.py:90
cloudra/base/skill_class.py:91
cloudra/base/skill_class.py:92
cloudra/base/skill_class.py:93
cloudra/base/skill_class.py:94
cloudra/base/skill_class.py:95
cloudra/base/skill_class.py:96
cloudra/base/skill_class.py:97
cloudra/base/skill_class.py:98
cloudra/base/skill_class.py:99
cloudra/base/tool_actor.py:1    """工具看到的调用者身份（ToolActorContext）。
cloudra/base/tool_actor.py:2
cloudra/base/tool_actor.py:3    工具（bash/read/write/dag_*/goal/todo/…）只需要「谁在调我」的少量信息：工作
cloudra/base/tool_actor.py:4    目录、会话与项目键、权限位、编排模式。此前它们直接 import
cloudra/base/tool_actor.py:5    ``cloudra.core.agent`` 的 ``Agent``——那个类带 LLM 调用能力，工具因此与已下线
```

```
cloudra/base/tool_actor.py:6    的执行栈耦合：字段被收窄成具体类还逼得执行器用 ``object.__setattr__`` 绕过
cloudra/base/tool_actor.py:7    pydantic 校验（见 ``runtime_v2/adapters.py``）。该执行栈（``Agent``/
cloudra/base/tool_actor.py:8    ``Coordinator``/``Follower`` 与 ``core/message_history.py``）已随评审 5.4
cloudra/base/tool_actor.py:9    删除，这里只剩工具真正需要的表面。
cloudra/base/tool_actor.py:10
cloudra/base/tool_actor.py:11   这里只声明**工具真正用到**的表面。生产实现是 ``runtime_v2.adapters`` 里包
cloudra/base/tool_actor.py:12   ``AgentSession`` 的那个对象；测试可用任意鸭子类型对象。
cloudra/base/tool_actor.py:13
cloudra/base/tool_actor.py:14   刻意不声明的字段：``agent_id`` 等 DAG 库里的历史 id（工具用
cloudra/base/tool_actor.py:15   ``getattr(..., default)`` 兜底读取，属于迁移边界，不是新工具参数的首选身份）。
cloudra/base/tool_actor.py:16   """
cloudra/base/tool_actor.py:17
cloudra/base/tool_actor.py:18   from __future__ import annotations
cloudra/base/tool_actor.py:19
cloudra/base/tool_actor.py:20   from typing import TYPE_CHECKING, Protocol, runtime_checkable
cloudra/base/tool_actor.py:21
cloudra/base/tool_actor.py:22   if TYPE_CHECKING:
cloudra/base/tool_actor.py:23       from cloudra.tools.core.permission_model import Permission
cloudra/base/tool_actor.py:24
cloudra/base/tool_actor.py:25
cloudra/base/tool_actor.py:26   @runtime_checkable
cloudra/base/tool_actor.py:27   class ToolActorContext(Protocol):
cloudra/base/tool_actor.py:28       """调用方 Agent 暴露给工具的只读身份（不含任何执行能力）。"""
cloudra/base/tool_actor.py:29
cloudra/base/tool_actor.py:30       #: 会话 id（v2 为 ``as_*``；历史数据可能是旧的 a-N 组合 id）。
cloudra/base/tool_actor.py:31       id: str
cloudra/base/tool_actor.py:32       #: 调用者显示名；v2 下即 role_id（``coordinator`` / ``small_mol`` / …）。
cloudra/base/tool_actor.py:33       name: str
cloudra/base/tool_actor.py:34       #: 调用者类型：``coordinator`` / ``supervisor`` / ``follower``（DAG actor 判据）。
cloudra/base/tool_actor.py:35       type: str
cloudra/base/tool_actor.py:36       #: 会话键（v2 = project_id）：todos/goal 等 per-agent 存储的分区键。
cloudra/base/tool_actor.py:37       conversation_id: str
cloudra/base/tool_actor.py:38       #: 工具解析相对路径的根目录（绝不回落进程 cwd）。
cloudra/base/tool_actor.py:39       working_directory: str
cloudra/base/tool_actor.py:40       #: 编排模式字符串（standard / dag_record / dag_guided）。
cloudra/base/tool_actor.py:41       orchestration_mode: str
cloudra/base/tool_actor.py:42       #: 会话 id 别名；同角色并行实例（swarm）据此隔离各自的 todos/goal。
cloudra/base/tool_actor.py:43       session_id: str
cloudra/base/tool_actor.py:44
cloudra/base/tool_actor.py:45       def permission_all(self) -> "Permission":
cloudra/base/tool_actor.py:46           """调用者的权限位（IntFlag 投影，供 bash 等工具判定命令类别）。"""
cloudra/base/tool_actor.py:47           ...
cloudra/base/tools_class.py:1   from contextlib import contextmanager
cloudra/base/tools_class.py:2   from contextvars import ContextVar
cloudra/base/tools_class.py:3   from typing import TYPE_CHECKING, Any, Awaitable, Callable, Optional
cloudra/base/tools_class.py:4
cloudra/base/tools_class.py:5   from pydantic import BaseModel
cloudra/base/tools_class.py:6   from pydantic_core import PydanticUndefined
cloudra/base/tools_class.py:7
cloudra/base/tools_class.py:8   from cloudra.base.session_class import Session
cloudra/base/tools_class.py:9   from cloudra.persistence.database import ConversationDB, get_database_path
cloudra/base/tools_class.py:10
```

```
cloudra/base/tools_class.py:11  # HACK: 这里原本缺少 import，属于历史遗留问题；get_tools_lst 定义在 tools/__init__.py
cloudra/base/tools_class.py:12  from cloudra.tools import get_tools_lst
cloudra/base/tools_class.py:13  from cloudra.tools.core.permission_model import Permission
cloudra/base/tools_class.py:14
cloudra/base/tools_class.py:15  if TYPE_CHECKING:
cloudra/base/tools_class.py:16      from cloudra.infra.yolo_guard import YoloGuard
cloudra/base/tools_class.py:17
cloudra/base/tools_class.py:18  # runtime_v2 的审批钩子：LegacyToolExecutor 在执行工具前总是设置；
cloudra/base/tools_class.py:19  # 未设置即生产外路径（v1/CLI/Flet 已删除），request_approval 直接抛错。
cloudra/base/tools_class.py:20  # 签名：async (**fields) -> tuple[bool, str]，字段同 request_approval 的关键字参数。
cloudra/base/tools_class.py:21  _APPROVAL_HANDLER: ContextVar[
cloudra/base/tools_class.py:22      Optional[Callable[..., Awaitable[tuple[bool, str]]]]
cloudra/base/tools_class.py:23  ] = ContextVar("cloudra_approval_handler", default=None)
cloudra/base/tools_class.py:24
cloudra/base/tools_class.py:25
cloudra/base/tools_class.py:26  @contextmanager
cloudra/base/tools_class.py:27  def use_approval_handler(handler: Callable[..., Awaitable[tuple[bool, str]]]):
cloudra/base/tools_class.py:28      """Scope a v2 approval handler to one tool execution (per-task isolation)."""
cloudra/base/tools_class.py:29      token = _APPROVAL_HANDLER.set(handler)
cloudra/base/tools_class.py:30      try:
cloudra/base/tools_class.py:31          yield
cloudra/base/tools_class.py:32      finally:
cloudra/base/tools_class.py:33          _APPROVAL_HANDLER.reset(token)
cloudra/base/tools_class.py:34
cloudra/base/tools_class.py:35
cloudra/base/tools_class.py:36  def _tool_db_path(subagent) -> str:
cloudra/base/tools_class.py:37      """根据 tool 关联的 Agent 返回对应 conversation 的数据库路径。"""
cloudra/base/tools_class.py:38      conv_id = getattr(subagent, 'conversation_id', 'default') if subagent else 'default'
cloudra/base/tools_class.py:39      return get_database_path(conv_id)
cloudra/base/tools_class.py:40
cloudra/base/tools_class.py:41
cloudra/base/tools_class.py:42  def get_tools() -> list[dict]:
cloudra/base/tools_class.py:43      """将工具类加载为 OpenAI function 格式的 dict 列表。"""
cloudra/base/tools_class.py:44      entries = get_tools_lst()
cloudra/base/tools_class.py:45      tools_list = []
cloudra/base/tools_class.py:46      for entry in entries:
cloudra/base/tools_class.py:47          tool_instance = entry.instantiate()
cloudra/base/tools_class.py:48          tools_list.append(tool_instance.tool_schema_json())
cloudra/base/tools_class.py:49      return tools_list
cloudra/base/tools_class.py:50
cloudra/base/tools_class.py:51
cloudra/base/tools_class.py:52  class input_schema(BaseModel):
cloudra/base/tools_class.py:53      '''
cloudra/base/tools_class.py:54      工具输入模式基类，定义工具需要的输入参数和格式。
cloudra/base/tools_class.py:55
cloudra/base/tools_class.py:56      子类用法示例：
cloudra/base/tools_class.py:57          class MyInput(input_schema):
cloudra/base/tools_class.py:58              query: str = Field(description="搜索关键词")
cloudra/base/tools_class.py:59              max_results: int = Field(default=10, description="最大返回条数")
cloudra/base/tools_class.py:60
cloudra/base/tools_class.py:61      通过 to_parameters_schema() 可自动生成符合 OpenAI Function Calling 规范的 parameters 字典。
cloudra/base/tools_class.py:62      '''
```

```
cloudra/base/tools_class.py:63
cloudra/base/tools_class.py:64      @classmethod
cloudra/base/tools_class.py:65      def to_parameters_schema(cls) -> dict:
cloudra/base/tools_class.py:66          '''
cloudra/base/tools_class.py:67          生成符合 OpenAI Function Calling 的 parameters 字典。
cloudra/base/tools_class.py:68          基于 Pydantic model_json_schema() 微调：
cloudra/base/tools_class.py:69          - properties / required 已由 Pydantic 根据字段声明自动生成
cloudra/base/tools_class.py:70          - 强制附加 additionalProperties: false，防止 LLM 传入未定义字段
cloudra/base/tools_class.py:71          '''
cloudra/base/tools_class.py:72          schema = cls.model_json_schema()
cloudra/base/tools_class.py:73          parameters = {
cloudra/base/tools_class.py:74              "type": "object",
cloudra/base/tools_class.py:75              "properties": schema.get("properties", {}),
cloudra/base/tools_class.py:76              "required": schema.get("required", []),
cloudra/base/tools_class.py:77              "additionalProperties": False,
cloudra/base/tools_class.py:78          }
cloudra/base/tools_class.py:79          if "$defs" in schema:
cloudra/base/tools_class.py:80              parameters["$defs"] = schema["$defs"]
cloudra/base/tools_class.py:81          return parameters
cloudra/base/tools_class.py:82
cloudra/base/tools_class.py:83      @classmethod
cloudra/base/tools_class.py:84      def from_args(cls, arg_dict: dict) -> "input_schema":
cloudra/base/tools_class.py:85          '''从 LLM 返回的 arguments dict 构造输入实例，带 Pydantic 类型校验。'''
cloudra/base/tools_class.py:86          return cls(**arg_dict)
cloudra/base/tools_class.py:87
cloudra/base/tools_class.py:88      def toClass(self, arg_dict: dict) -> "input_schema":
cloudra/base/tools_class.py:89          '''
cloudra/base/tools_class.py:90          兼容旧接口，供 reAct.py 调用。
cloudra/base/tools_class.py:91          实际调用链通常为 tool.input_schema.toClass(tool_args)，此时 self 为 input_schema 子类本
    身。
cloudra/base/tools_class.py:92          '''
cloudra/base/tools_class.py:93          # 当通过类调用时（如 TextReader_input_schema.toClass(args)），self 即为类对象
cloudra/base/tools_class.py:94          if isinstance(self, type) and issubclass(self, input_schema):
cloudra/base/tools_class.py:95              return self(**arg_dict)
cloudra/base/tools_class.py:96          return self.__class__(**arg_dict)
cloudra/base/tools_class.py:97
cloudra/base/tools_class.py:98      def toJson(self) -> dict:
cloudra/base/tools_class.py:99          '''实例序列化为 dict，用于日志或上下文回传。'''
cloudra/base/tools_class.py:100         return self.model_dump(mode="json")
cloudra/base/tools_class.py:101
cloudra/base/tools_class.py:102
cloudra/base/tools_class.py:103 class output_schema(BaseModel):
cloudra/base/tools_class.py:104     '''
cloudra/base/tools_class.py:105     工具输出模式基类，定义工具输出的格式。
cloudra/base/tools_class.py:106     子类按普通 Pydantic Model 声明字段即可。
cloudra/base/tools_class.py:107     '''
cloudra/base/tools_class.py:108
cloudra/base/tools_class.py:109     def toJson(self) -> dict:
cloudra/base/tools_class.py:110         return self.model_dump(mode="json")
cloudra/base/tools_class.py:111
cloudra/base/tools_class.py:112
cloudra/base/tools_class.py:113 class Tool(BaseModel):
cloudra/base/tools_class.py:114     '''
```

```
115 cloudra/base/tools_class.py:115     工具基类。所有具体工具应继承此类。
116 cloudra/base/tools_class.py:116
117 cloudra/base/tools_class.py:117     关于 input_schema / output_schema:
118 cloudra/base/tools_class.py:118     - 这里存放的是具体的 schema 类（如 TextReader_input_schema），而非实例。
119 cloudra/base/tools_class.py:119     - 使用 Any 类型并开启 arbitrary_types_allowed，避免 Pydantic 对类对象做字段校验。
120 cloudra/base/tools_class.py:120     - 子模块中覆盖时必须带类型注解，例如:
121 cloudra/base/tools_class.py:121           input_schema: Any = TextReader_input_schema
122 cloudra/base/tools_class.py:122     '''
123 cloudra/base/tools_class.py:123     model_config = {"arbitrary_types_allowed": True}
124 cloudra/base/tools_class.py:124
125 cloudra/base/tools_class.py:125     # 唯一标识符
126 cloudra/base/tools_class.py:126     tool_id: str = ''
127 cloudra/base/tools_class.py:127
128 cloudra/base/tools_class.py:128     # 工具名称（即 OpenAI function.name）
129 cloudra/base/tools_class.py:129     tool_name: str = ''
130 cloudra/base/tools_class.py:130
131 cloudra/base/tools_class.py:131     # 工具描述。改造后应仅保留【功能描述】+【使用场景】，不再包含参数细节。
132 cloudra/base/tools_class.py:132     # 参数说明通过 input_schema 的 Field(description="...") 提供，自动生成到 parameters 中。
133 cloudra/base/tools_class.py:133     tool_description: str = ''
134 cloudra/base/tools_class.py:134
135 cloudra/base/tools_class.py:135     # 工具的输入模式类（存放类对象，非实例）
136 cloudra/base/tools_class.py:136     input_schema: Any = input_schema
137 cloudra/base/tools_class.py:137
138 cloudra/base/tools_class.py:138     # 工具的输出模式类（存放类对象，非实例）
139 cloudra/base/tools_class.py:139     output_schema: Any = output_schema
140 cloudra/base/tools_class.py:140
141 cloudra/base/tools_class.py:141     # 引用调用该工具的 Agent 实例
142 cloudra/base/tools_class.py:142     subagent: Any = None
143 cloudra/base/tools_class.py:143
144 cloudra/base/tools_class.py:144     def is_read(self) -> bool:
145 cloudra/base/tools_class.py:145         return False
146 cloudra/base/tools_class.py:146
147 cloudra/base/tools_class.py:147     def is_write(self) -> bool:
148 cloudra/base/tools_class.py:148         return False
149 cloudra/base/tools_class.py:149
150 cloudra/base/tools_class.py:150     def is_manage(self) -> bool:
151 cloudra/base/tools_class.py:151         return False
152 cloudra/base/tools_class.py:152
153 cloudra/base/tools_class.py:153     def is_script(self) -> bool:
154 cloudra/base/tools_class.py:154         return False
155 cloudra/base/tools_class.py:155
156 cloudra/base/tools_class.py:156     def is_destructive(self) -> bool:
157 cloudra/base/tools_class.py:157         return False
158 cloudra/base/tools_class.py:158
159 cloudra/base/tools_class.py:159     def tool_permission_all(self) -> Permission:
160 cloudra/base/tools_class.py:160         """Return Permission flags for this tool."""
161 cloudra/base/tools_class.py:161         perms = Permission.NONE
162 cloudra/base/tools_class.py:162         if self.is_read():
163 cloudra/base/tools_class.py:163             perms |= Permission.READ
164 cloudra/base/tools_class.py:164         if self.is_write():
165 cloudra/base/tools_class.py:165             perms |= Permission.WRITE
166 cloudra/base/tools_class.py:166         if self.is_manage():
```

```
cloudra/base/tools_class.py:167             perms |= Permission.MANAGE
cloudra/base/tools_class.py:168         if self.is_script():
cloudra/base/tools_class.py:169             perms |= Permission.SCRIPT
cloudra/base/tools_class.py:170         if self.is_destructive():
cloudra/base/tools_class.py:171             perms |= Permission.DESTRUCTIVE
cloudra/base/tools_class.py:172         return perms
cloudra/base/tools_class.py:173
cloudra/base/tools_class.py:174     async def tool_call(self, user_input: input_schema, session: Session) -> output_schema:
cloudra/base/tools_class.py:175         '''
cloudra/base/tools_class.py:176         调用工具。子类必须重写此方法。
cloudra/base/tools_class.py:177         执行结果以 output_schema 实例返回。
cloudra/base/tools_class.py:178         '''
cloudra/base/tools_class.py:179         raise NotImplementedError("子类必须实现 tool_call 方法")
cloudra/base/tools_class.py:180
cloudra/base/tools_class.py:181     async def safe_call(self, user_input: input_schema, session: Session) -> output_schema:
cloudra/base/tools_class.py:182         '''
cloudra/base/tools_class.py:183         带有通用异常捕获的工具调用包装器。
cloudra/base/tools_class.py:184         工具执行失败时抛出包含工具名的 RuntimeError，由 reAct.act_result() 统一捕获并回传给 LLM。
cloudra/base/tools_class.py:185         消息以 [Tool error] 前缀 + 工具名 + 恢复指引成形：react 层识别该前缀后不再二次套壳。
cloudra/base/tools_class.py:186         '''
cloudra/base/tools_class.py:187         try:
cloudra/base/tools_class.py:188             return await self.tool_call(user_input, session)
cloudra/base/tools_class.py:189         except Exception as e:
cloudra/base/tools_class.py:190             error_msg = (
cloudra/base/tools_class.py:191                 f"[Tool error] {self.tool_name}: {str(e)}. "
cloudra/base/tools_class.py:192                 "Fix the arguments or use a different tool, then retry."
cloudra/base/tools_class.py:193             )
cloudra/base/tools_class.py:194             raise RuntimeError(error_msg) from e
cloudra/base/tools_class.py:195
cloudra/base/tools_class.py:196     async def ensure_safety(
cloudra/base/tools_class.py:197         self,
cloudra/base/tools_class.py:198         session: Session,
cloudra/base/tools_class.py:199         details: str,
cloudra/base/tools_class.py:200         tool_args: dict | None = None,
cloudra/base/tools_class.py:201     ) -> tuple[bool, str]:
cloudra/base/tools_class.py:202         '''
cloudra/base/tools_class.py:203         安全检查：如果工具被标记为 destructive，则必须经过审批。
cloudra/base/tools_class.py:204         默认通过数据库等待用户审批；若启用了 YOLO 模式，则调用独立 Agent 自动判断。
cloudra/base/tools_class.py:205         返回 (是否通过, 拒绝原因)。通过时拒绝原因为空字符串。
cloudra/base/tools_class.py:206         '''
cloudra/base/tools_class.py:207         from cloudra.infra.yolo_guard import YoloGuard, is_yolo_enabled
cloudra/base/tools_class.py:208
cloudra/base/tools_class.py:209         risk_summary = next(
cloudra/base/tools_class.py:210             (
cloudra/base/tools_class.py:211                 line.partition(":")[2].strip()
cloudra/base/tools_class.py:212                 for line in details.splitlines()
cloudra/base/tools_class.py:213                 if line.lower().startswith("action:") or line.startswith("【动作】")
cloudra/base/tools_class.py:214             ),
cloudra/base/tools_class.py:215             details.splitlines()[0] if details.splitlines() else "Dangerous tool call",
cloudra/base/tools_class.py:216         )
cloudra/base/tools_class.py:217
cloudra/base/tools_class.py:218         if is_yolo_enabled():
```

```
cloudra/base/tools_class.py:219             guard = YoloGuard()
cloudra/base/tools_class.py:220             return await self._yolo_judge(session, tool_args or {}, guard, risk_summary=
risk_summary)
cloudra/base/tools_class.py:221
cloudra/base/tools_class.py:222         db_path = _tool_db_path(self.subagent)
cloudra/base/tools_class.py:223         candidate_diff = ""
cloudra/base/tools_class.py:224         for marker in ("Unified Diff:\n", "【Unified Diff】:\n"):
cloudra/base/tools_class.py:225             if marker in details:
cloudra/base/tools_class.py:226                 candidate_diff = details.partition(marker)[2]
cloudra/base/tools_class.py:227                 break
cloudra/base/tools_class.py:228
cloudra/base/tools_class.py:229         return await self.request_approval(
cloudra/base/tools_class.py:230             db_path,
cloudra/base/tools_class.py:231             session.id,
cloudra/base/tools_class.py:232             details,
cloudra/base/tools_class.py:233             tool_name=self.tool_name,
cloudra/base/tools_class.py:234             normalized_arguments=tool_args or {},
cloudra/base/tools_class.py:235             risk_summary=risk_summary,
cloudra/base/tools_class.py:236             candidate_diff=candidate_diff,
cloudra/base/tools_class.py:237         )
cloudra/base/tools_class.py:238
cloudra/base/tools_class.py:239     async def record_session_event(
cloudra/base/tools_class.py:240         self,
cloudra/base/tools_class.py:241         session: Session,
cloudra/base/tools_class.py:242         event_type: str,
cloudra/base/tools_class.py:243         data: dict,
cloudra/base/tools_class.py:244     ) -> None:
cloudra/base/tools_class.py:245         """尽力记录工具事件：遥测失败不能反向破坏工具主流程。"""
cloudra/base/tools_class.py:246         from cloudra.services import session_store as _ss
cloudra/base/tools_class.py:247
cloudra/base/tools_class.py:248         try:
cloudra/base/tools_class.py:249             db = ConversationDB.from_path(_tool_db_path(self.subagent))
cloudra/base/tools_class.py:250             await _ss.arecord_event(db, session.id, event_type, data)
cloudra/base/tools_class.py:251         except Exception:
cloudra/base/tools_class.py:252             pass
cloudra/base/tools_class.py:253
cloudra/base/tools_class.py:254     async def _yolo_judge(
cloudra/base/tools_class.py:255         self,
cloudra/base/tools_class.py:256         session: Session,
cloudra/base/tools_class.py:257         tool_args: dict,
cloudra/base/tools_class.py:258         guard: 'YoloGuard',
cloudra/base/tools_class.py:259         risk_summary: str = "",
cloudra/base/tools_class.py:260     ) -> tuple[bool, str]:
cloudra/base/tools_class.py:261         """调用 YoloGuard 做自动判断。"""
cloudra/base/tools_class.py:262         approved, reason = await guard.judge(
cloudra/base/tools_class.py:263             self.tool_name, tool_args, self.subagent, session, risk_summary=risk_summary
cloudra/base/tools_class.py:264         )
cloudra/base/tools_class.py:265         if approved:
cloudra/base/tools_class.py:266             print("\033[92m[安全审计/YOLO]\033[0m 自动审批通过。")
cloudra/base/tools_class.py:267             return True, ""
cloudra/base/tools_class.py:268         else:
cloudra/base/tools_class.py:269             reason_text = reason or "YOLO Agent 认为该操作存在风险"
cloudra/base/tools_class.py:270             print(f"\033[91m[安全审计/YOLO]\033[0m 自动审批拒绝。原因: {reason_text}")
```

```
271 cloudra/base/tools_class.py:271             return False, reason_text
272 cloudra/base/tools_class.py:272
273 cloudra/base/tools_class.py:273     async def request_approval(
274 cloudra/base/tools_class.py:274         self,
275 cloudra/base/tools_class.py:275         db_path: str,
276 cloudra/base/tools_class.py:276         session_id: str,
277 cloudra/base/tools_class.py:277         details: str,
278 cloudra/base/tools_class.py:278         *,
279 cloudra/base/tools_class.py:279         tool_name: str = "",
280 cloudra/base/tools_class.py:280         normalized_arguments: dict | None = None,
281 cloudra/base/tools_class.py:281         risk_summary: str = "",
282 cloudra/base/tools_class.py:282         candidate_diff: str = "",
283 cloudra/base/tools_class.py:283     ) -> tuple[bool, str]:
284 cloudra/base/tools_class.py:284         handler = _APPROVAL_HANDLER.get()
285 cloudra/base/tools_class.py:285         if handler is None:
286 cloudra/base/tools_class.py:286             # v1 时代"SQLite + 事件总线等待"的审批回退已随旧服务层删除。
287 cloudra/base/tools_class.py:287             raise RuntimeError(
288 cloudra/base/tools_class.py:288                 "no approval handler installed: runtime_v2 LegacyToolExecutor always sets
    one"
289 cloudra/base/tools_class.py:289             )
290 cloudra/base/tools_class.py:290         # runtime_v2 上下文：审批请求与等待由内核 broker 承担，
291 cloudra/base/tools_class.py:291         # 不碰 conversation SQLite。
292 cloudra/base/tools_class.py:292         return await handler(
293 cloudra/base/tools_class.py:293             details=details,
294 cloudra/base/tools_class.py:294             tool_name=tool_name,
295 cloudra/base/tools_class.py:295             normalized_arguments=normalized_arguments or {},
296 cloudra/base/tools_class.py:296             risk_summary=risk_summary,
297 cloudra/base/tools_class.py:297             candidate_diff=candidate_diff,
298 cloudra/base/tools_class.py:298         )
299 cloudra/base/tools_class.py:299
300 cloudra/base/tools_class.py:300     @classmethod
301 cloudra/base/tools_class.py:301     def _get_field_default(cls, field_name: str):
302 cloudra/base/tools_class.py:302         '''获取类字段的默认值，兼容 Pydantic v2（类级别无法直接访问普通字段）。'''
303 cloudra/base/tools_class.py:303         field = cls.model_fields.get(field_name)
304 cloudra/base/tools_class.py:304         if field and field.default is not PydanticUndefined:
305 cloudra/base/tools_class.py:305             return field.default
306 cloudra/base/tools_class.py:306         return None
307 cloudra/base/tools_class.py:307
308 cloudra/base/tools_class.py:308     def tool_schema_json(self) -> dict:
309 cloudra/base/tools_class.py:309         '''
310 cloudra/base/tools_class.py:310         返回标准 OpenAI function 格式，可直接用于 client.chat.completions.create(..., tools
    =[...])。
311 cloudra/base/tools_class.py:311         优先读取实例级别的 tool_description，使动态生成的描述能够生效。
312 cloudra/base/tools_class.py:312         '''
313 cloudra/base/tools_class.py:313         # HACK: 这里用 getattr + or fallback 链是因为 Pydantic v2 的类字段默认值在实例上可能
314 cloudra/base/tools_class.py:314         # 被 shadow，但某些情况下实例属性又还没覆盖。动态 description（如 Bash 的权限列表、
315 cloudra/base/tools_class.py:315         # GetSkill 的 skills 列表）必须在 __init__ 之后才能确定，因此必须是实例方法而非类方法。
316 cloudra/base/tools_class.py:316         # 如果后续有更好方式（如 descriptor / property），可替换掉这个 ugly fallback。
317 cloudra/base/tools_class.py:317         description = getattr(self, 'tool_description', '') or self._get_field_default('
    tool_description') or ''
318 cloudra/base/tools_class.py:318         return {
319 cloudra/base/tools_class.py:319             "type": "function",
320 cloudra/base/tools_class.py:320             "function": {
321 cloudra/base/tools_class.py:321                 "name": getattr(self, 'tool_name', '') or self._get_field_default('
    tool_name') or '',
322 cloudra/base/tools_class.py:322                 "description": description,
```

```
cloudra/base/tools_class.py:323                "parameters": (self._get_field_default('input_schema') or input_schema).to_parameters_schema(),
cloudra/base/tools_class.py:324            }
cloudra/base/tools_class.py:325        }
cloudra/base/tools_class.py:326
cloudra/base/tools_class.py:327    def toJson(self) -> dict:
cloudra/base/tools_class.py:328        '''
cloudra/base/tools_class.py:329        实例级序列化，主要用于日志或调试。
cloudra/base/tools_class.py:330        若需获取 OpenAI schema，请使用类方法 tool_schema_json()。
cloudra/base/tools_class.py:331        '''
cloudra/base/tools_class.py:332        return {
cloudra/base/tools_class.py:333            "tool_id": self.tool_id,
cloudra/base/tools_class.py:334            "tool_name": self.tool_name,
cloudra/base/tools_class.py:335            "tool_description": self.tool_description,
cloudra/base/tools_class.py:336        }
cloudra/base/tools_class.py:337
cloudra/base/tools_class.py:338    @classmethod
cloudra/base/tools_class.py:339    def build_input(cls, arg_dict: dict) -> input_schema:
cloudra/base/tools_class.py:340        '''
cloudra/base/tools_class.py:341        将 LLM 返回的参数字典转换为 input_schema 实例。
cloudra/base/tools_class.py:342        供 reAct.py 调用，替代原有的 tool.input_schema.toClass(tool_args)。
cloudra/base/tools_class.py:343        '''
cloudra/base/tools_class.py:344        schema_cls = cls._get_field_default('input_schema') or input_schema
cloudra/base/tools_class.py:345        return schema_cls.from_args(arg_dict)
cloudra/benchmark/__init__.py:1 """Benchmark module: benchmark task schema, task generators, and the scoring pipeline.
cloudra/benchmark/__init__.py:2
cloudra/benchmark/__init__.py:3 Public surface:
cloudra/benchmark/__init__.py:4 - `schema`: TaskSpec / GoldSpec / RubricSpec / mechanism-graph models.
cloudra/benchmark/__init__.py:5 - `gen_easy_tasks`: generator turning ideapool/reaction_candidates/easy.md
cloudra/benchmark/__init__.py:6   into ideapool/tasks/easy/*.json.
cloudra/benchmark/__init__.py:7 - `graph_similarity`: chemical-parameterized GED between two mechanism graphs.
cloudra/benchmark/__init__.py:8 - `integrity`: sandbox forensic scan + evidence binding (Layer 1).
cloudra/benchmark/__init__.py:9 - `selfconsistency`: physical self-consistency checks (Layer 2).
cloudra/benchmark/__init__.py:10    - `accuracy`: Gaussian-tolerance barrier scoring (Layer 3).
cloudra/benchmark/__init__.py:11    - `method_policy`: method-whitelist interception.
cloudra/benchmark/__init__.py:12    - `runner`: independent benchmark runner with its own data directory.
cloudra/benchmark/__init__.py:13    """
cloudra/benchmark/__init__.py:14
cloudra/benchmark/__init__.py:15    from cloudra.benchmark.schema import (  # noqa: F401
cloudra/benchmark/__init__.py:16        AccuracyRubric,
cloudra/benchmark/__init__.py:17        BondChangeSet,
cloudra/benchmark/__init__.py:18        ExpectedOutcome,
cloudra/benchmark/__init__.py:19        GoldMechanismGraph,
cloudra/benchmark/__init__.py:20        GoldSpec,
cloudra/benchmark/__init__.py:21        GraphSimilarityRubric,
cloudra/benchmark/__init__.py:22        ImpossibleCaseRubric,
cloudra/benchmark/__init__.py:23        IntegrityRubric,
cloudra/benchmark/__init__.py:24        MechanismEdge,
cloudra/benchmark/__init__.py:25        MechanismNode,
cloudra/benchmark/__init__.py:26        MethodPolicy,
cloudra/benchmark/__init__.py:27        NodeRole,
cloudra/benchmark/__init__.py:28        PromptSpec,
cloudra/benchmark/__init__.py:29        RubricSpec,
```

```
cloudra/benchmark/__init__.py:30        SelfConsistencyRubric,
cloudra/benchmark/__init__.py:31        SystemSpec,
cloudra/benchmark/__init__.py:32        TaskSpec,
cloudra/benchmark/__init__.py:33        Tier,
cloudra/benchmark/__init__.py:34    )
cloudra/benchmark/__main__.py:1 """python -m cloudra.benchmark — run the benchmark task bank.
cloudra/benchmark/__main__.py:2
cloudra/benchmark/__main__.py:3 Default is a dry-run (no-op backend: sandboxes stay empty, every task scores
cloudra/benchmark/__main__.py:4 0 through the real scoring pipeline — useful for checking the harness itself).
cloudra/benchmark/__main__.py:5 `--backend runtime_v2` runs each task on the real in-process runtime_v2 kernel
cloudra/benchmark/__main__.py:6 with the method whitelist enforced.
cloudra/benchmark/__main__.py:7
cloudra/benchmark/__main__.py:8 每题除三层评分外还按拒答分类（answered/refused/failed，见 refusal.py）；
cloudra/benchmark/__main__.py:9 批次结束后打印跨档拒答 precision/recall（ground truth =
cloudra/benchmark/__main__.py:10    task.rubric.expected_outcome）。`--tasks-dir` 可重复传入多个目录——拒答
cloudra/benchmark/__main__.py:11    统计需要"应当拒答"（impossible）与"应当作答"（其他档/兄弟题）同批对照。
cloudra/benchmark/__main__.py:12
cloudra/benchmark/__main__.py:13    Example:
cloudra/benchmark/__main__.py:14        python -m cloudra.benchmark --backend runtime_v2 --data-dir bench_out --only C1,D3
cloudra/benchmark/__main__.py:15        python -m cloudra.benchmark --tasks-dir ideapool/tasks/easy --tasks-dir ideapool/
tasks/impossible
cloudra/benchmark/__main__.py:16    """
cloudra/benchmark/__main__.py:17
cloudra/benchmark/__main__.py:18    from __future__ import annotations
cloudra/benchmark/__main__.py:19
cloudra/benchmark/__main__.py:20    import argparse
cloudra/benchmark/__main__.py:21    import os
cloudra/benchmark/__main__.py:22    import sys
cloudra/benchmark/__main__.py:23    from pathlib import Path
cloudra/benchmark/__main__.py:24    from typing import Optional
cloudra/benchmark/__main__.py:25
cloudra/benchmark/__main__.py:26
cloudra/benchmark/__main__.py:27    def _fmt(x: Optional[float]) -> str:
cloudra/benchmark/__main__.py:28        return f"{x:.3f}" if x is not None else "—"
cloudra/benchmark/__main__.py:29
cloudra/benchmark/__main__.py:30
cloudra/benchmark/__main__.py:31    def main(argv: list[str] | None = None) -> int:
cloudra/benchmark/__main__.py:32        from cloudra.benchmark.refusal import RunOutcome, refusal_stats, should_refuse
cloudra/benchmark/__main__.py:33        from cloudra.benchmark.runner import BenchmarkRunner, load_tasks
cloudra/benchmark/__main__.py:34
cloudra/benchmark/__main__.py:35        ap = argparse.ArgumentParser(prog="cloudra.benchmark")
cloudra/benchmark/__main__.py:36        ap.add_argument("--tasks-dir", type=Path, action="append", default=None,
cloudra/benchmark/__main__.py:37                        help="task JSON dir; repeatable for cross-tier refusal stats "
cloudra/benchmark/__main__.py:38                             "(default: ideapool/tasks/easy)")
cloudra/benchmark/__main__.py:39        ap.add_argument("--data-dir", type=Path, default=Path("benchmark_data"),
cloudra/benchmark/__main__.py:40                        help="benchmark data dir (sandboxes + benchmark.db)")
cloudra/benchmark/__main__.py:41        ap.add_argument("--backend", choices=["none", "runtime_v2"], default="none")
cloudra/benchmark/__main__.py:42        ap.add_argument("--only", default="", help="comma-separated task ids (default: all)
")
cloudra/benchmark/__main__.py:43        ap.add_argument("--timeout", type=float, default=7200.0, help="per-task run timeout
, seconds")
cloudra/benchmark/__main__.py:44        ap.add_argument("--concurrency", type=int, default=1,
cloudra/benchmark/__main__.py:45                        help="parallel tasks (threads, per-task event loop + sandbox;
default 1 = serial)")
cloudra/benchmark/__main__.py:46        ap.add_argument("--api-pool", type=Path, default=None,
cloudra/benchmark/__main__.py:47                        help='JSON endpoint pool for anti-429: [{"base_url","api_key","
label"}, ...]; '
```

```
cloudra/benchmark/__main__.py:48                            "each LLM call picks a random endpoint, 429 fails over to the next")
cloudra/benchmark/__main__.py:49        args = ap.parse_args(argv)
cloudra/benchmark/__main__.py:50
cloudra/benchmark/__main__.py:51        tasks = []
cloudra/benchmark/__main__.py:52        for d in (args.tasks_dir or [Path("ideapool/tasks/easy")]):
cloudra/benchmark/__main__.py:53            tasks.extend(load_tasks(d))
cloudra/benchmark/__main__.py:54        if args.only:
cloudra/benchmark/__main__.py:55            wanted = {t.strip() for t in args.only.split(",") if t.strip()}
cloudra/benchmark/__main__.py:56            tasks = [t for t in tasks if t.task_id in wanted]
cloudra/benchmark/__main__.py:57        if not tasks:
cloudra/benchmark/__main__.py:58            print("no tasks matched", file=sys.stderr)
cloudra/benchmark/__main__.py:59            return 2
cloudra/benchmark/__main__.py:60
cloudra/benchmark/__main__.py:61        backend = None
cloudra/benchmark/__main__.py:62        if args.backend == "runtime_v2":
cloudra/benchmark/__main__.py:63            # 运行时数据隔离必须在首次读取全局配置前完成：config_file 独立于
cloudra/benchmark/__main__.py:64            # data_dir、LLM 凭据不受影响。YAML 里的 data_dir 键会覆盖环境变量，
cloudra/benchmark/__main__.py:65            # 所以两处都钉死。
cloudra/benchmark/__main__.py:66            isolated = (args.data_dir / "runtime").resolve()
cloudra/benchmark/__main__.py:67            os.environ["CLOUDRA_DATA_DIR"] = str(isolated)
cloudra/benchmark/__main__.py:68            from cloudra.infra.config import get_config
cloudra/benchmark/__main__.py:69
cloudra/benchmark/__main__.py:70            get_config().data_dir = isolated
cloudra/benchmark/__main__.py:71            llm = None
cloudra/benchmark/__main__.py:72            if args.api_pool is not None:
cloudra/benchmark/__main__.py:73                from cloudra.benchmark.api_pool import PooledLLMProvider, load_endpoints
cloudra/benchmark/__main__.py:74
cloudra/benchmark/__main__.py:75                llm = PooledLLMProvider(load_endpoints(args.api_pool))
cloudra/benchmark/__main__.py:76            from cloudra.benchmark.backend import RuntimeV2Backend
cloudra/benchmark/__main__.py:77
cloudra/benchmark/__main__.py:78            backend = RuntimeV2Backend(
cloudra/benchmark/__main__.py:79                run_timeout_s=args.timeout,
cloudra/benchmark/__main__.py:80                llm=llm,
cloudra/benchmark/__main__.py:81                store_path=args.data_dir / "runtime" / "runtime-v2.db",
cloudra/benchmark/__main__.py:82            )
cloudra/benchmark/__main__.py:83
cloudra/benchmark/__main__.py:84        # CLI 每次调用也是一个批次：batches 行 + 溯源快照（模型/模式/插件）+
cloudra/benchmark/__main__.py:85        # runs 行带 batch_id——CLI 与 Web 结果在同一 schema 下可并排比较。
cloudra/benchmark/__main__.py:86        from cloudra.benchmark.runner import provenance_config
cloudra/benchmark/__main__.py:87        from cloudra.runtime_v2.schemas import new_id
cloudra/benchmark/__main__.py:88
cloudra/benchmark/__main__.py:89        batch_id = new_id("batch")
cloudra/benchmark/__main__.py:90        runner = BenchmarkRunner(args.data_dir, backend=backend, batch_id=batch_id)
cloudra/benchmark/__main__.py:91        runner.db.record_batch(batch_id, "running", {
cloudra/benchmark/__main__.py:92            **provenance_config("cli", tasks),
cloudra/benchmark/__main__.py:93            "tasks_dirs": [str(d) for d in (args.tasks_dir or [Path("ideapool/tasks/easy")])],
cloudra/benchmark/__main__.py:94            "task_ids": [t.task_id for t in tasks],
cloudra/benchmark/__main__.py:95            "concurrency": max(1, args.concurrency),
cloudra/benchmark/__main__.py:96            "run_timeout_s": args.timeout,
cloudra/benchmark/__main__.py:97        })
cloudra/benchmark/__main__.py:98        status = "done"
cloudra/benchmark/__main__.py:99        try:
```

```
cloudra/benchmark/__main__.py:100           results = runner.run_all(tasks, concurrency=max(1, args.concurrency))
cloudra/benchmark/__main__.py:101       except BaseException:
cloudra/benchmark/__main__.py:102           status = "error"
cloudra/benchmark/__main__.py:103           raise
cloudra/benchmark/__main__.py:104       finally:
cloudra/benchmark/__main__.py:105           runner.db.finish_batch(batch_id, status)
cloudra/benchmark/__main__.py:106       for r in results:
cloudra/benchmark/__main__.py:107           print(
cloudra/benchmark/__main__.py:108               f"{r.task_id:>6}  final={r.final_score:.3f}  outcome={r.outcome:<8} "
cloudra/benchmark/__main__.py:109               f"integrity={int(r.integrity_passed)} graph={r.graph_score:.3f} "
cloudra/benchmark/__main__.py:110               f"selfcons={int(r.selfconsistency_passed)} accuracy={r.accuracy_score:.3f}"
cloudra/benchmark/__main__.py:111           )
cloudra/benchmark/__main__.py:112
cloudra/benchmark/__main__.py:113       stats = refusal_stats(
cloudra/benchmark/__main__.py:114           (should_refuse(t), r.outcome == RunOutcome.REFUSED.value)
cloudra/benchmark/__main__.py:115           for t, r in zip(tasks, results)
cloudra/benchmark/__main__.py:116       )
cloudra/benchmark/__main__.py:117       print(
cloudra/benchmark/__main__.py:118           f"\nrefusal: TP={stats.tp} FP={stats.fp} FN={stats.fn} TN={stats.tn}  "
cloudra/benchmark/__main__.py:119           f"precision={_fmt(stats.precision)} recall={_fmt(stats.recall)} f1={_fmt(stats.
    f1)}"
cloudra/benchmark/__main__.py:120       )
cloudra/benchmark/__main__.py:121       return 0
cloudra/benchmark/__main__.py:122
cloudra/benchmark/__main__.py:123
cloudra/benchmark/__main__.py:124   if __name__ == "__main__":
cloudra/benchmark/__main__.py:125       sys.exit(main())
cloudra/benchmark/accuracy.py:1 """Layer 3 — accuracy: 以交付物为准的逐边能垒/性质评分（不读任何 LLM 散文）。
cloudra/benchmark/accuracy.py:2
cloudra/benchmark/accuracy.py:3 口径（与题库 query 协议一致）：gold 机理图**逐边**承载参考答案能垒；评分对
cloudra/benchmark/accuracy.py:4 **每条带参考值的 gold 边**在交付 evidence 里按 edge_id 找对应步，要求两条独立
cloudra/benchmark/accuracy.py:5 来源**都存在且一致**：
cloudra/benchmark/accuracy.py:6
cloudra/benchmark/accuracy.py:7 - ``log_barrier_kcal``：harness 从沙箱内绑定日志推出的值（``barrier_evidence``）；
cloudra/benchmark/accuracy.py:8 - ``declared_barrier_kcal``：交付 ``mechanism_graph.json`` 里该边声明的
cloudra/benchmark/accuracy.py:9   ``barrier_kcal_mol``。
cloudra/benchmark/accuracy.py:10
cloudra/benchmark/accuracy.py:11    该边缺日志推算、缺声明、或声明在**任何合法组合**下都得不到支撑且证据也对
cloudra/benchmark/accuracy.py:12    不上参考 → 该边 0 分并 fail-loud 归零整题（绝不用最终消息里的数字兜底）。
cloudra/benchmark/accuracy.py:13    声明支撑得住（或虽支撑不住、但交付证据里另有与参考一致的组合——即 agent
cloudra/benchmark/accuracy.py:14    选了另一个构象/参考系定义、而正确的计算已在它的日志里）→ 走正常数值路径或
cloudra/benchmark/accuracy.py:15    按「答案定义差异」固定小扣分（`_DEFINITION_MISMATCH_EDGE_SCORE`），**不归零**
cloudra/benchmark/accuracy.py:16    （用户口径：这是答案定义问题，不是科学错误）。一致后与 gold 边值比，
cloudra/benchmark/accuracy.py:17    用高斯容差评分；逐边分数按 gold 边 ``weight`` 加权平均（权重越大 = 该边越
cloudra/benchmark/accuracy.py:18    容易算准/越可核实）。汇总后仍按 ``max_penalty_fraction`` 封顶。
cloudra/benchmark/accuracy.py:19
cloudra/benchmark/accuracy.py:20    若 gold 机理图不存在任何带参考值的边 → accuracy 不评分（final_score=1.0），
cloudra/benchmark/accuracy.py:21    沿用旧 null 行为。
cloudra/benchmark/accuracy.py:22
cloudra/benchmark/accuracy.py:23    性质锚点（``GoldSpec.reference_properties``，键带单位）：读交付图物种节点的
cloudra/benchmark/accuracy.py:24    ``properties`` 按键比对——数值型走高斯容差（σ 查模块级显式常量表
cloudra/benchmark/accuracy.py:25    ``_PROPERTY_SIGMAS``，键名子串匹配，表外数值键 fail-loud 抛错逼显式登记）；
cloudra/benchmark/accuracy.py:26    字符串型（state_ordering 等）小写去空白后精确相等。多性质题取各键均值。
```

```
cloudra/benchmark/accuracy.py:27
cloudra/benchmark/accuracy.py:28    hard 档定性选择性（``score_qualitative_selectivity``）：任务带
cloudra/benchmark/accuracy.py:29    ``gold.selectivity.favored_channel`` 时替代 accuracy 槽位（composite 的 0.3
cloudra/benchmark/accuracy.py:30    权重给选择性分）——交付边中恰好一条 ``favored: true`` 且其 ``channel`` 等于
cloudra/benchmark/accuracy.py:31    gold favored_channel → 1.0，否则 0.0；barrier spread 数值不评分。
cloudra/benchmark/accuracy.py:32    """
cloudra/benchmark/accuracy.py:33
cloudra/benchmark/accuracy.py:34    from __future__ import annotations
cloudra/benchmark/accuracy.py:35
cloudra/benchmark/accuracy.py:36    import math
cloudra/benchmark/accuracy.py:37    from dataclasses import dataclass, field
cloudra/benchmark/accuracy.py:38    from typing import TYPE_CHECKING, Optional
cloudra/benchmark/accuracy.py:39
cloudra/benchmark/accuracy.py:40    from cloudra.benchmark.schema import (
cloudra/benchmark/accuracy.py:41        AccuracyRubric,
cloudra/benchmark/accuracy.py:42        GoldMechanismGraph,
cloudra/benchmark/accuracy.py:43        GoldSpec,
cloudra/benchmark/accuracy.py:44        SelectivityReference,
cloudra/benchmark/accuracy.py:45    )
cloudra/benchmark/accuracy.py:46
cloudra/benchmark/accuracy.py:47    if TYPE_CHECKING:
cloudra/benchmark/accuracy.py:48        from cloudra.benchmark.barrier_evidence import BarrierEvidence
cloudra/benchmark/accuracy.py:49
cloudra/benchmark/accuracy.py:50    # 性质锚点的高斯容差表（键名子串匹配，首个命中生效；表外数值键 fail-loud）。
cloudra/benchmark/accuracy.py:51    # 新增性质键必须在此显式登记 σ——没有默认兜底值。
cloudra/benchmark/accuracy.py:52    _PROPERTY_SIGMAS = (
cloudra/benchmark/accuracy.py:53        ("bond_length_angstrom", 0.02),
cloudra/benchmark/accuracy.py:54        ("cm-1", 50.0),
cloudra/benchmark/accuracy.py:55        ("kcal_mol", 2.0),
cloudra/benchmark/accuracy.py:56    )
cloudra/benchmark/accuracy.py:57
cloudra/benchmark/accuracy.py:58    # 「答案定义差异」的逐边记分：证据里已经含有与参考一致的组合（agent 只是
cloudra/benchmark/accuracy.py:59    # 报/选了另一个构象或参考系定义），属定义层面的不一致而非科学错误——按此
cloudra/benchmark/accuracy.py:60    # 固定小扣分计入加权，绝不走 fail-loud 归零（用户口径）。
cloudra/benchmark/accuracy.py:61    _DEFINITION_MISMATCH_EDGE_SCORE = 0.9
cloudra/benchmark/accuracy.py:62
cloudra/benchmark/accuracy.py:63
cloudra/benchmark/accuracy.py:64    @dataclass
cloudra/benchmark/accuracy.py:65    class AccuracyReport:
cloudra/benchmark/accuracy.py:66        enabled: bool = True
cloudra/benchmark/accuracy.py:67        log_barrier_kcal: float | None = None
cloudra/benchmark/accuracy.py:68        declared_barrier_kcal: float | None = None
cloudra/benchmark/accuracy.py:69        rds_edge_id: str = ""
cloudra/benchmark/accuracy.py:70        sigma_kcal: float = 3.0
cloudra/benchmark/accuracy.py:71        gaussian_score: float = 0.0          # in [0, 1], 1 = exact match
cloudra/benchmark/accuracy.py:72        capped_penalty: float = 0.0          # penalty actually applied after cap
cloudra/benchmark/accuracy.py:73        final_score: float = 0.0             # 1 - capped_penalty（缺证据/不一致直接边 0）
cloudra/benchmark/accuracy.py:74        # 逐边能垒得分：edge_id -> 高斯分（空 = 非能垒模式或本层未评分）
cloudra/benchmark/accuracy.py:75        edge_scores: dict[str, float] = field(default_factory=dict)
cloudra/benchmark/accuracy.py:76        # 性质锚点模式：逐键得分（空 = 能垒模式或本层未评分）
cloudra/benchmark/accuracy.py:77        property_scores: dict[str, float] = field(default_factory=dict)
cloudra/benchmark/accuracy.py:78        notes: list[str] = field(default_factory=list)
```

```
cloudra/benchmark/accuracy.py:79
cloudra/benchmark/accuracy.py:80
cloudra/benchmark/accuracy.py:81    def gaussian_barrier_score(agent: float, gold: float, sigma: float) -> float:
cloudra/benchmark/accuracy.py:82        """exp(-(agent-gold)^2 / (2 sigma^2)) in [0, 1]."""
cloudra/benchmark/accuracy.py:83        if sigma <= 0:
cloudra/benchmark/accuracy.py:84            return 1.0 if agent == gold else 0.0
cloudra/benchmark/accuracy.py:85        d = agent - gold
cloudra/benchmark/accuracy.py:86        return math.exp(-(d * d) / (2.0 * sigma * sigma))
cloudra/benchmark/accuracy.py:87
cloudra/benchmark/accuracy.py:88
cloudra/benchmark/accuracy.py:89    def _effective_sigma(rubric: AccuracyRubric, gold_barrier_kcal: float) -> float:
cloudra/benchmark/accuracy.py:90        """逐边对 gold 比较的高斯宽度：小能垒放宽（rubric 小能垒口径），
cloudra/benchmark/accuracy.py:91        大能垒用基准 sigma。只用于对 gold 的比较，不动内部一致性核对。"""
cloudra/benchmark/accuracy.py:92        if gold_barrier_kcal <= rubric.small_barrier_threshold_kcal:
cloudra/benchmark/accuracy.py:93            return max(rubric.sigma_kcal_mol, rubric.small_barrier_sigma_kcal)
cloudra/benchmark/accuracy.py:94        return rubric.sigma_kcal_mol
cloudra/benchmark/accuracy.py:95
cloudra/benchmark/accuracy.py:96
cloudra/benchmark/accuracy.py:97    def _property_sigma(key: str) -> float:
cloudra/benchmark/accuracy.py:98        """性质键的高斯 σ：显式常量表子串匹配；表外数值键 fail-loud（逼显式登记）。"""
cloudra/benchmark/accuracy.py:99        for token, sigma in _PROPERTY_SIGMAS:
cloudra/benchmark/accuracy.py:100           if token in key:
cloudra/benchmark/accuracy.py:101               return sigma
cloudra/benchmark/accuracy.py:102       raise ValueError(
cloudra/benchmark/accuracy.py:103           f"no sigma registered for property key {key!r}; "
cloudra/benchmark/accuracy.py:104           "extend _PROPERTY_SIGMAS explicitly"
cloudra/benchmark/accuracy.py:105       )
cloudra/benchmark/accuracy.py:106
cloudra/benchmark/accuracy.py:107
cloudra/benchmark/accuracy.py:108   def _normalize_text(value: str) -> str:
cloudra/benchmark/accuracy.py:109       return " ".join(value.lower().split())
cloudra/benchmark/accuracy.py:110
cloudra/benchmark/accuracy.py:111
cloudra/benchmark/accuracy.py:112   def _score_properties(
cloudra/benchmark/accuracy.py:113       delivered_graph: Optional[GoldMechanismGraph],
cloudra/benchmark/accuracy.py:114       gold_properties: dict[str, float | str],
cloudra/benchmark/accuracy.py:115       rep: AccuracyReport,
cloudra/benchmark/accuracy.py:116   ) -> AccuracyReport:
cloudra/benchmark/accuracy.py:117       """性质锚点模式：交付图物种节点的 `properties` 逐键对 gold 比对取均值。"""
cloudra/benchmark/accuracy.py:118       scores: dict[str, float] = {}
cloudra/benchmark/accuracy.py:119       for key, gold_val in gold_properties.items():
cloudra/benchmark/accuracy.py:120           if delivered_graph is None:
cloudra/benchmark/accuracy.py:121               rep.notes.append(f"property {key}: no delivered graph; scored 0")
cloudra/benchmark/accuracy.py:122               scores[key] = 0.0
cloudra/benchmark/accuracy.py:123               continue
cloudra/benchmark/accuracy.py:124           declared = [
cloudra/benchmark/accuracy.py:125               (n.id, n.properties[key])
cloudra/benchmark/accuracy.py:126               for n in delivered_graph.nodes
cloudra/benchmark/accuracy.py:127               if key in n.properties
cloudra/benchmark/accuracy.py:128           ]
cloudra/benchmark/accuracy.py:129           if not declared:
cloudra/benchmark/accuracy.py:130               rep.notes.append(
```

```
cloudra/benchmark/accuracy.py:131                 f"property {key}: not declared on any species node; scored 0"
cloudra/benchmark/accuracy.py:132             )
cloudra/benchmark/accuracy.py:133             scores[key] = 0.0
cloudra/benchmark/accuracy.py:134             continue
cloudra/benchmark/accuracy.py:135         if len(declared) > 1:
cloudra/benchmark/accuracy.py:136             rep.notes.append(
cloudra/benchmark/accuracy.py:137                 f"property {key}: declared on multiple nodes "
cloudra/benchmark/accuracy.py:138                 f"{[nid for nid, _ in declared]}; using {declared[0][0]} "
cloudra/benchmark/accuracy.py:139                 "(first in graph order)"
cloudra/benchmark/accuracy.py:140             )
cloudra/benchmark/accuracy.py:141         node_id, val = declared[0]
cloudra/benchmark/accuracy.py:142         if isinstance(gold_val, str):
cloudra/benchmark/accuracy.py:143             if not isinstance(val, str):
cloudra/benchmark/accuracy.py:144                 rep.notes.append(
cloudra/benchmark/accuracy.py:145                     f"property {key}: gold expects a string, node {node_id} "
cloudra/benchmark/accuracy.py:146                     f"declared {type(val).__name__}; scored 0"
cloudra/benchmark/accuracy.py:147                 )
cloudra/benchmark/accuracy.py:148                 scores[key] = 0.0
cloudra/benchmark/accuracy.py:149             else:
cloudra/benchmark/accuracy.py:150                 scores[key] = (
cloudra/benchmark/accuracy.py:151                     1.0 if _normalize_text(val) == _normalize_text(gold_val) else 0.0
cloudra/benchmark/accuracy.py:152                 )
cloudra/benchmark/accuracy.py:153                 rep.notes.append(
cloudra/benchmark/accuracy.py:154                     f"property {key}: node {node_id} declares {val!r} vs gold "
cloudra/benchmark/accuracy.py:155                     f"{gold_val!r} -> {scores[key]}"
cloudra/benchmark/accuracy.py:156                 )
cloudra/benchmark/accuracy.py:157         else:
cloudra/benchmark/accuracy.py:158             if isinstance(val, bool) or not isinstance(val, (int, float)):
cloudra/benchmark/accuracy.py:159                 rep.notes.append(
cloudra/benchmark/accuracy.py:160                     f"property {key}: gold expects a number, node {node_id} "
cloudra/benchmark/accuracy.py:161                     f"declared {val!r}; scored 0"
cloudra/benchmark/accuracy.py:162                 )
cloudra/benchmark/accuracy.py:163                 scores[key] = 0.0
cloudra/benchmark/accuracy.py:164             else:
cloudra/benchmark/accuracy.py:165                 sigma = _property_sigma(key)
cloudra/benchmark/accuracy.py:166                 scores[key] = gaussian_barrier_score(float(val), float(gold_val), sigma
    )
cloudra/benchmark/accuracy.py:167                 rep.notes.append(
cloudra/benchmark/accuracy.py:168                     f"property {key}: node {node_id} declares {val} vs gold "
cloudra/benchmark/accuracy.py:169                     f"{gold_val} (sigma {sigma}) -> {scores[key]:.4f}"
cloudra/benchmark/accuracy.py:170                 )
cloudra/benchmark/accuracy.py:171     rep.property_scores = scores
cloudra/benchmark/accuracy.py:172     rep.gaussian_score = sum(scores.values()) / len(scores)
cloudra/benchmark/accuracy.py:173     rep.final_score = rep.gaussian_score
cloudra/benchmark/accuracy.py:174     return rep
cloudra/benchmark/accuracy.py:175
cloudra/benchmark/accuracy.py:176
cloudra/benchmark/accuracy.py:177 def score_qualitative_selectivity(
cloudra/benchmark/accuracy.py:178     delivered_graph: Optional[GoldMechanismGraph],
cloudra/benchmark/accuracy.py:179     selectivity: SelectivityReference,
cloudra/benchmark/accuracy.py:180 ) -> tuple[float, str]:
cloudra/benchmark/accuracy.py:181     """hard 档定性选择性：交付边中恰好一条 favored=true 且 channel 命中 gold
cloudra/benchmark/accuracy.py:182     favored_channel → 1.0；其余（无 favored 声明、多条 favored、channel 错）→ 0.0。
```

```
cloudra/benchmark/accuracy.py:183       barrier spread 数值不评分。"""
cloudra/benchmark/accuracy.py:184       if delivered_graph is None or not delivered_graph.edges:
cloudra/benchmark/accuracy.py:185           return 0.0, (
cloudra/benchmark/accuracy.py:186               "qualitative selectivity: no delivered graph edges; favored channel "
cloudra/benchmark/accuracy.py:187               "not declared"
cloudra/benchmark/accuracy.py:188           )
cloudra/benchmark/accuracy.py:189       favored = [e for e in delivered_graph.edges if e.favored is True]
cloudra/benchmark/accuracy.py:190       if not favored:
cloudra/benchmark/accuracy.py:191           return 0.0, (
cloudra/benchmark/accuracy.py:192               "qualitative selectivity: no edge declares `favored: true`; scored 0"
cloudra/benchmark/accuracy.py:193           )
cloudra/benchmark/accuracy.py:194       if len(favored) > 1:
cloudra/benchmark/accuracy.py:195           return 0.0, (
cloudra/benchmark/accuracy.py:196               "qualitative selectivity: multiple edges declare `favored: true` "
cloudra/benchmark/accuracy.py:197               f"({[e.id for e in favored]}); the contract requires exactly one; scored 0"
cloudra/benchmark/accuracy.py:198           )
cloudra/benchmark/accuracy.py:199       edge = favored[0]
cloudra/benchmark/accuracy.py:200       if edge.channel == selectivity.favored_channel:
cloudra/benchmark/accuracy.py:201           return 1.0, (
cloudra/benchmark/accuracy.py:202               f"qualitative selectivity: favored channel {edge.channel!r} matches "
cloudra/benchmark/accuracy.py:203               "the reference; scored 1"
cloudra/benchmark/accuracy.py:204           )
cloudra/benchmark/accuracy.py:205       return 0.0, (
cloudra/benchmark/accuracy.py:206           f"qualitative selectivity: favored channel {edge.channel!r} != reference "
cloudra/benchmark/accuracy.py:207           f"{selectivity.favored_channel!r}; scored 0"
cloudra/benchmark/accuracy.py:208       )
cloudra/benchmark/accuracy.py:209
cloudra/benchmark/accuracy.py:210
cloudra/benchmark/accuracy.py:211   def _find_matching_step(
cloudra/benchmark/accuracy.py:212       gold_edge,
cloudra/benchmark/accuracy.py:213       gold_nodes: dict,
cloudra/benchmark/accuracy.py:214       steps: list,
cloudra/benchmark/accuracy.py:215       delivered_graph: Optional[GoldMechanismGraph],
cloudra/benchmark/accuracy.py:216       cfg,
cloudra/benchmark/accuracy.py:217       match_threshold: float,
cloudra/benchmark/accuracy.py:218   ):
cloudra/benchmark/accuracy.py:219       """按**物种对应**找 gold 边的交付步（edge id 是题库内部约定，agent 不可知，
cloudra/benchmark/accuracy.py:220       绝不参与匹配）：交付边的 from/to 端点物种与 gold 边**同向**端点分别匹配
cloudra/benchmark/accuracy.py:221       （species_substitution_cost ≤ 阈值，与 Layer 1/graph GED 同口径；反向步
cloudra/benchmark/accuracy.py:222       能垒口径不同不匹配）。多候选按 bond_changes 对称差最小、端点代价最小、
cloudra/benchmark/accuracy.py:223       edge id 字典序取（确定性）。"""
cloudra/benchmark/accuracy.py:224       from cloudra.benchmark.graph_similarity import species_substitution_cost
cloudra/benchmark/accuracy.py:225
cloudra/benchmark/accuracy.py:226       if delivered_graph is None:
cloudra/benchmark/accuracy.py:227           return None
cloudra/benchmark/accuracy.py:228       delivered_nodes = {n.id: n for n in delivered_graph.nodes}
cloudra/benchmark/accuracy.py:229       delivered_edges = {e.id: e for e in delivered_graph.edges}
cloudra/benchmark/accuracy.py:230       gold_from = gold_nodes.get(gold_edge.from_node)
cloudra/benchmark/accuracy.py:231       gold_to = gold_nodes.get(gold_edge.to_node)
cloudra/benchmark/accuracy.py:232       if gold_from is None or gold_to is None:
cloudra/benchmark/accuracy.py:233           return None
cloudra/benchmark/accuracy.py:234       gold_pairs = gold_edge.bond_changes.pairs()
```

```
    candidates: list[tuple[int, float, str, object]] = []
    for step in steps:
        d_edge = delivered_edges.get(step.edge_id)
        if d_edge is None:
            continue
        d_from = delivered_nodes.get(d_edge.from_node)
        d_to = delivered_nodes.get(d_edge.to_node)
        if d_from is None or d_to is None:
            continue
        cost_from = species_substitution_cost(gold_from.smiles, d_from.smiles, cfg)
        cost_to = species_substitution_cost(gold_to.smiles, d_to.smiles, cfg)
        if cost_from > match_threshold or cost_to > match_threshold:
            continue
        symdiff = len(gold_pairs ^ d_edge.bond_changes.pairs())
        candidates.append((symdiff, cost_from + cost_to, step.edge_id, step))
    if not candidates:
        return None
    candidates.sort(key=lambda item: (item[0], item[1], item[2]))
    return candidates[0][3]


def score_accuracy(
    *,
    evidence: "BarrierEvidence",
    gold: GoldSpec,
    rubric: AccuracyRubric | None = None,
    delivered_graph: Optional[GoldMechanismGraph] = None,
    cfg=None,
    match_threshold: float = 0.2,
) -> AccuracyReport:
    """逐边核对（每条 gold 带参考值的边都要"声明得到日志支撑"），再逐边对
    gold 比、按 gold 边 weight 加权平均。gold.reference_properties 非空时走性质
    锚点模式（与能垒模式互斥）。"""
    rubric = rubric or AccuracyRubric()
    rds = evidence.rds()
    rep = AccuracyReport(
        enabled=rubric.enabled,
        sigma_kcal=rubric.sigma_kcal_mol,
        log_barrier_kcal=rds.barrier_kcal_mol if rds is not None else None,
        declared_barrier_kcal=rds.declared_kcal_mol if rds is not None else None,
        rds_edge_id=rds.edge_id if rds is not None else "",
    )

    if not rubric.enabled:
        rep.gaussian_score = 1.0
        rep.final_score = 1.0
        rep.notes.append("accuracy disabled for this task; scored on self-consistency
only")
        return rep
    if gold.reference_properties:
        return _score_properties(delivered_graph, gold.reference_properties, rep)
    ref_edges = [
        e for e in gold.mechanism_graph.edges if e.barrier_kcal_mol is not None
```

```
287 cloudra/benchmark/accuracy.py:287       ]
288 cloudra/benchmark/accuracy.py:288       if not ref_edges:
289 cloudra/benchmark/accuracy.py:289           rep.gaussian_score = 1.0
290 cloudra/benchmark/accuracy.py:290           rep.final_score = 1.0
291 cloudra/benchmark/accuracy.py:291           rep.notes.append("no gold edge carries a reference barrier; accuracy not scored
    ")
292 cloudra/benchmark/accuracy.py:292           return rep
293 cloudra/benchmark/accuracy.py:293
294 cloudra/benchmark/accuracy.py:294       # --- 交付物双来源门禁：逐边核对（任一参考边缺失/不一致 -> 整题 fail-loud 记 0） ---
295 cloudra/benchmark/accuracy.py:295       from cloudra.benchmark.graph_similarity import GedConfig
296 cloudra/benchmark/accuracy.py:296
297 cloudra/benchmark/accuracy.py:297       cfg = cfg or GedConfig()
298 cloudra/benchmark/accuracy.py:298       gold_nodes = {n.id: n for n in gold.mechanism_graph.nodes}
299 cloudra/benchmark/accuracy.py:299       # 全零权重无法归一：退回逐边等权（避免除零；正常题库不出现）
300 cloudra/benchmark/accuracy.py:300       equal_fallback = sum(e.weight for e in ref_edges) <= 0
301 cloudra/benchmark/accuracy.py:301       if equal_fallback:
302 cloudra/benchmark/accuracy.py:302           rep.notes.append(
303 cloudra/benchmark/accuracy.py:303               "all reference-edge weights are zero; falling back to equal weighting"
304 cloudra/benchmark/accuracy.py:304           )
305 cloudra/benchmark/accuracy.py:305       weights = {e.id: (1.0 if equal_fallback else e.weight) for e in ref_edges}
306 cloudra/benchmark/accuracy.py:306       weight_total = sum(weights.values())
307 cloudra/benchmark/accuracy.py:307
308 cloudra/benchmark/accuracy.py:308       failures: list[str] = []
309 cloudra/benchmark/accuracy.py:309       weighted_sum = 0.0
310 cloudra/benchmark/accuracy.py:310       for edge in ref_edges:
311 cloudra/benchmark/accuracy.py:311           step = _find_matching_step(
312 cloudra/benchmark/accuracy.py:312               edge, gold_nodes, evidence.steps, delivered_graph, cfg, match_threshold
313 cloudra/benchmark/accuracy.py:313           )
314 cloudra/benchmark/accuracy.py:314           if step is None:
315 cloudra/benchmark/accuracy.py:315               rep.edge_scores[edge.id] = 0.0
316 cloudra/benchmark/accuracy.py:316               failures.append(
317 cloudra/benchmark/accuracy.py:317                   f"gold edge {edge.id} carries a reference barrier but no delivered "
318 cloudra/benchmark/accuracy.py:318                   "step matches its endpoint species"
319 cloudra/benchmark/accuracy.py:319               )
320 cloudra/benchmark/accuracy.py:320               continue
321 cloudra/benchmark/accuracy.py:321           if step.edge_id != edge.id:
322 cloudra/benchmark/accuracy.py:322               rep.notes.append(
323 cloudra/benchmark/accuracy.py:323                   f"gold edge {edge.id} matched delivered step {step.edge_id} "
324 cloudra/benchmark/accuracy.py:324                   "by endpoint species"
325 cloudra/benchmark/accuracy.py:325               )
326 cloudra/benchmark/accuracy.py:326           if step.declared_kcal_mol is None:
327 cloudra/benchmark/accuracy.py:327               rep.edge_scores[edge.id] = 0.0
328 cloudra/benchmark/accuracy.py:328               failures.append(
329 cloudra/benchmark/accuracy.py:329                   "delivered mechanism_graph.json declares no barrier_kcal_mol on "
330 cloudra/benchmark/accuracy.py:330                   f"step edge {edge.id}"
331 cloudra/benchmark/accuracy.py:331               )
332 cloudra/benchmark/accuracy.py:332               continue
333 cloudra/benchmark/accuracy.py:333           declared = step.declared_kcal_mol
334 cloudra/benchmark/accuracy.py:334           # 该边全部合法组合（TS 日志 × 起点极小值日志；含策略选中的那组）。
335 cloudra/benchmark/accuracy.py:335           # 声明支撑得住哪一组、哪一组能对上参考，决定这条边怎么算分：
336 cloudra/benchmark/accuracy.py:336           # 支撑得住 → 正常数值路径；支撑不住但证据里有能对参考的 → 答案定义/
337 cloudra/benchmark/accuracy.py:337           # 构象指认差异（小扣分，不是科学错误）；两者皆无 → 无据声明 fail-loud。
338 cloudra/benchmark/accuracy.py:338           candidates = evidence.alternatives.get(step.edge_id) or [step]
```

```
cloudra/benchmark/accuracy.py:339           sigma_eff = _effective_sigma(rubric, edge.barrier_kcal_mol)
cloudra/benchmark/accuracy.py:340           supported = [
cloudra/benchmark/accuracy.py:341               c for c in candidates
cloudra/benchmark/accuracy.py:342               if abs(declared - c.barrier_kcal_mol) <= rubric.sigma_kcal_mol
cloudra/benchmark/accuracy.py:343           ]
cloudra/benchmark/accuracy.py:344           matches_ref = [
cloudra/benchmark/accuracy.py:345               c for c in candidates
cloudra/benchmark/accuracy.py:346               if abs(c.barrier_kcal_mol - edge.barrier_kcal_mol) <= sigma_eff
cloudra/benchmark/accuracy.py:347           ]
cloudra/benchmark/accuracy.py:348
cloudra/benchmark/accuracy.py:349           if supported:
cloudra/benchmark/accuracy.py:350               same_as_ref = any(
cloudra/benchmark/accuracy.py:351                   (c.ts_path, c.base_path) in {(m.ts_path, m.base_path) for m in
    matches_ref}
cloudra/benchmark/accuracy.py:352                   for c in supported
cloudra/benchmark/accuracy.py:353               )
cloudra/benchmark/accuracy.py:354               if matches_ref and not same_as_ref:
cloudra/benchmark/accuracy.py:355                   rep.edge_scores[edge.id] = _DEFINITION_MISMATCH_EDGE_SCORE
cloudra/benchmark/accuracy.py:356                   weighted_sum += weights[edge.id] * _DEFINITION_MISMATCH_EDGE_SCORE
cloudra/benchmark/accuracy.py:357                   ref = min(
cloudra/benchmark/accuracy.py:358                       matches_ref,
cloudra/benchmark/accuracy.py:359                       key=lambda c: (abs(c.barrier_kcal_mol - edge.barrier_kcal_mol),
cloudra/benchmark/accuracy.py:360                                      c.ts_path, c.base_path),
cloudra/benchmark/accuracy.py:361                   )
cloudra/benchmark/accuracy.py:362                   rep.notes.append(
cloudra/benchmark/accuracy.py:363                       f"edge {edge.id}: declared {declared:.3f} is supported by "
cloudra/benchmark/accuracy.py:364                       f"{supported[0].ts_path} / {supported[0].base_path}, but the "
cloudra/benchmark/accuracy.py:365                       f"delivered evidence also contains a combination consistent "
cloudra/benchmark/accuracy.py:366                       f"with the reference ({ref.barrier_kcal_mol:.3f} vs gold "
cloudra/benchmark/accuracy.py:367                       f"{edge.barrier_kcal_mol:.3f}, {ref.ts_path} / {ref.base_path}) "
cloudra/benchmark/accuracy.py:368                       "-> answer-definition mismatch, scored "
cloudra/benchmark/accuracy.py:369                       f"{_DEFINITION_MISMATCH_EDGE_SCORE}"
cloudra/benchmark/accuracy.py:370                   )
cloudra/benchmark/accuracy.py:371                   continue
cloudra/benchmark/accuracy.py:372               chosen = step if step in supported else min(
cloudra/benchmark/accuracy.py:373                   supported,
cloudra/benchmark/accuracy.py:374                   key=lambda c: (abs(declared - c.barrier_kcal_mol), c.ts_path, c.
    base_path),
cloudra/benchmark/accuracy.py:375               )
cloudra/benchmark/accuracy.py:376               g = gaussian_barrier_score(chosen.barrier_kcal_mol, edge.barrier_kcal_mol,
    sigma_eff)
cloudra/benchmark/accuracy.py:377               rep.edge_scores[edge.id] = g
cloudra/benchmark/accuracy.py:378               weighted_sum += weights[edge.id] * g
cloudra/benchmark/accuracy.py:379               rep.notes.append(
cloudra/benchmark/accuracy.py:380                   f"edge {edge.id}: log-derived {chosen.barrier_kcal_mol:.3f} vs gold "
cloudra/benchmark/accuracy.py:381                   f"{edge.barrier_kcal_mol:.3f} (sigma {sigma_eff:.1f}) -> {g:.4f} "
cloudra/benchmark/accuracy.py:382                   f"(weight {weights[edge.id]})"
cloudra/benchmark/accuracy.py:383               )
cloudra/benchmark/accuracy.py:384               continue
cloudra/benchmark/accuracy.py:385
cloudra/benchmark/accuracy.py:386           if matches_ref:
cloudra/benchmark/accuracy.py:387               ref = min(
cloudra/benchmark/accuracy.py:388                   matches_ref,
cloudra/benchmark/accuracy.py:389                   key=lambda c: (abs(c.barrier_kcal_mol - edge.barrier_kcal_mol),
cloudra/benchmark/accuracy.py:390                                  c.ts_path, c.base_path),
```

```
cloudra/benchmark/accuracy.py:391                )
cloudra/benchmark/accuracy.py:392                rep.edge_scores[edge.id] = _DEFINITION_MISMATCH_EDGE_SCORE
cloudra/benchmark/accuracy.py:393                weighted_sum += weights[edge.id] * _DEFINITION_MISMATCH_EDGE_SCORE
cloudra/benchmark/accuracy.py:394                rep.notes.append(
cloudra/benchmark/accuracy.py:395                    f"edge {edge.id}: declared {declared:.3f} is not supported by any "
cloudra/benchmark/accuracy.py:396                    "delivered log combination, but the evidence contains one "
cloudra/benchmark/accuracy.py:397                    f"consistent with the reference ({ref.barrier_kcal_mol:.3f} vs gold "
cloudra/benchmark/accuracy.py:398                    f"{edge.barrier_kcal_mol:.3f}, {ref.ts_path} / {ref.base_path}) "
cloudra/benchmark/accuracy.py:399                    "-> answer-definition mismatch, scored "
cloudra/benchmark/accuracy.py:400                    f"{_DEFINITION_MISMATCH_EDGE_SCORE}"
cloudra/benchmark/accuracy.py:401                )
cloudra/benchmark/accuracy.py:402                continue
cloudra/benchmark/accuracy.py:403
cloudra/benchmark/accuracy.py:404            rep.edge_scores[edge.id] = 0.0
cloudra/benchmark/accuracy.py:405            failures.append(
cloudra/benchmark/accuracy.py:406                "declared vs log-derived barrier disagreement on edge "
cloudra/benchmark/accuracy.py:407                f"{edge.id} beyond sigma {rubric.sigma_kcal_mol}: declared "
cloudra/benchmark/accuracy.py:408                f"{declared:.3f} vs logs {step.barrier_kcal_mol:.3f}"
cloudra/benchmark/accuracy.py:409            )
cloudra/benchmark/accuracy.py:410
cloudra/benchmark/accuracy.py:411        if failures:
cloudra/benchmark/accuracy.py:412            # fail-loud：任一参考边的声明/推算缺失或不一致 -> 整题 accuracy 记 0，
cloudra/benchmark/accuracy.py:413            # 不给部分分，也不套 max_penalty_fraction 封顶。
cloudra/benchmark/accuracy.py:414            rep.notes.extend(f"{note}; scored 0" for note in failures)
cloudra/benchmark/accuracy.py:415            rep.gaussian_score = 0.0
cloudra/benchmark/accuracy.py:416            rep.capped_penalty = 1.0
cloudra/benchmark/accuracy.py:417            rep.final_score = 0.0
cloudra/benchmark/accuracy.py:418            return rep
cloudra/benchmark/accuracy.py:419
cloudra/benchmark/accuracy.py:420        rep.gaussian_score = weighted_sum / weight_total
cloudra/benchmark/accuracy.py:421        raw_penalty = 1.0 - rep.gaussian_score
cloudra/benchmark/accuracy.py:422        rep.capped_penalty = min(raw_penalty, rubric.max_penalty_fraction)
cloudra/benchmark/accuracy.py:423        rep.final_score = 1.0 - rep.capped_penalty
cloudra/benchmark/accuracy.py:424        if raw_penalty > rubric.max_penalty_fraction:
cloudra/benchmark/accuracy.py:425            rep.notes.append(
cloudra/benchmark/accuracy.py:426                f"raw penalty {raw_penalty:.3f} capped at {rubric.max_penalty_fraction}"
cloudra/benchmark/accuracy.py:427            )
cloudra/benchmark/accuracy.py:428        return rep
cloudra/benchmark/api_pool.py:1 """API endpoint pool for benchmark runs.
cloudra/benchmark/api_pool.py:2
cloudra/benchmark/api_pool.py:3 Anti-429 for parallel benchmark execution: every LLM call picks a random
cloudra/benchmark/api_pool.py:4 endpoint from the pool (spreading load), and if the call still exhausts the
cloudra/benchmark/api_pool.py:5 client-level backoff retries with a rate-limit error, the next endpoint takes
cloudra/benchmark/api_pool.py:6 over (no repeats within one call).
cloudra/benchmark/api_pool.py:7
cloudra/benchmark/api_pool.py:8 Scope by design: pool members share the *model* from the agent's settings —
cloudra/benchmark/api_pool.py:9 only the transport (base_url / api_key) is pinned per endpoint. Empty fields
cloudra/benchmark/api_pool.py:10    fall back to the agent's settings, so a pool can be just a list of extra keys
cloudra/benchmark/api_pool.py:11    for the same gateway. Non-429 errors are not retried here: they fail the turn
cloudra/benchmark/api_pool.py:12    (the runner's own turn-retry handles transient crashes as before).
cloudra/benchmark/api_pool.py:13    """
cloudra/benchmark/api_pool.py:14
```

```
cloudra/benchmark/api_pool.py:15    from __future__ import annotations
cloudra/benchmark/api_pool.py:16
cloudra/benchmark/api_pool.py:17    import contextvars
cloudra/benchmark/api_pool.py:18    import json
cloudra/benchmark/api_pool.py:19    import logging
cloudra/benchmark/api_pool.py:20    import random
cloudra/benchmark/api_pool.py:21    from dataclasses import dataclass
cloudra/benchmark/api_pool.py:22    from pathlib import Path
cloudra/benchmark/api_pool.py:23    from typing import Any, Optional
cloudra/benchmark/api_pool.py:24
cloudra/benchmark/api_pool.py:25    from cloudra.runtime_v2.adapters import LegacyLLMProvider
cloudra/benchmark/api_pool.py:26
cloudra/benchmark/api_pool.py:27    logger = logging.getLogger(__name__)
cloudra/benchmark/api_pool.py:28
cloudra/benchmark/api_pool.py:29
cloudra/benchmark/api_pool.py:30    @dataclass(frozen=True)
cloudra/benchmark/api_pool.py:31    class Endpoint:
cloudra/benchmark/api_pool.py:32        """One pool member; empty fields fall back to the agent's settings."""
cloudra/benchmark/api_pool.py:33
cloudra/benchmark/api_pool.py:34        base_url: str = ""
cloudra/benchmark/api_pool.py:35        api_key: str = ""
cloudra/benchmark/api_pool.py:36        label: str = ""  # 日志用：空则取 base_url
cloudra/benchmark/api_pool.py:37
cloudra/benchmark/api_pool.py:38        def display(self) -> str:
cloudra/benchmark/api_pool.py:39            return self.label or self.base_url or "(settings default)"
cloudra/benchmark/api_pool.py:40
cloudra/benchmark/api_pool.py:41
cloudra/benchmark/api_pool.py:42    # 当前调用钉住的端点。contextvars 按 asyncio task / 线程隔离——并行跑题时
cloudra/benchmark/api_pool.py:43    # 共享同一个 provider 实例也不会串端点。
cloudra/benchmark/api_pool.py:44    _current_endpoint: contextvars.ContextVar[Optional[Endpoint]] = contextvars.ContextVar(
cloudra/benchmark/api_pool.py:45        "benchmark_api_pool_endpoint", default=None
cloudra/benchmark/api_pool.py:46    )
cloudra/benchmark/api_pool.py:47
cloudra/benchmark/api_pool.py:48
cloudra/benchmark/api_pool.py:49    def load_endpoints(path: Path | str) -> list[Endpoint]:
cloudra/benchmark/api_pool.py:50        """Load a pool JSON: [{"base_url": ..., "api_key": ..., "label": ...}, ...]."""
cloudra/benchmark/api_pool.py:51        raw = json.loads(Path(path).read_text(encoding="utf-8"))
cloudra/benchmark/api_pool.py:52        if not isinstance(raw, list) or not raw:
cloudra/benchmark/api_pool.py:53            raise ValueError(f"api pool file must be a non-empty JSON list: {path}")
cloudra/benchmark/api_pool.py:54        endpoints = []
cloudra/benchmark/api_pool.py:55        for i, item in enumerate(raw):
cloudra/benchmark/api_pool.py:56            if not isinstance(item, dict):
cloudra/benchmark/api_pool.py:57                raise ValueError(f"api pool entry #{i} must be an object: {item!r}")
cloudra/benchmark/api_pool.py:58            endpoints.append(
cloudra/benchmark/api_pool.py:59                Endpoint(
cloudra/benchmark/api_pool.py:60                    base_url=str(item.get("base_url") or ""),
cloudra/benchmark/api_pool.py:61                    api_key=str(item.get("api_key") or ""),
cloudra/benchmark/api_pool.py:62                    label=str(item.get("label") or ""),
cloudra/benchmark/api_pool.py:63                )
cloudra/benchmark/api_pool.py:64            )
cloudra/benchmark/api_pool.py:65        return endpoints
cloudra/benchmark/api_pool.py:66
```

```
cloudra/benchmark/api_pool.py:67
cloudra/benchmark/api_pool.py:68    def _is_rate_limit(exc: BaseException) -> bool:
cloudra/benchmark/api_pool.py:69        try:
cloudra/benchmark/api_pool.py:70            import openai
cloudra/benchmark/api_pool.py:71        except Exception:  # pragma: no cover - openai 是核心依赖
cloudra/benchmark/api_pool.py:72            return False
cloudra/benchmark/api_pool.py:73        return isinstance(exc, openai.RateLimitError)
cloudra/benchmark/api_pool.py:74
cloudra/benchmark/api_pool.py:75
cloudra/benchmark/api_pool.py:76    class PooledLLMProvider(LegacyLLMProvider):
cloudra/benchmark/api_pool.py:77        """LegacyLLMProvider whose transport endpoint is pinned per call from a pool."""
cloudra/benchmark/api_pool.py:78
cloudra/benchmark/api_pool.py:79        def __init__(self, endpoints: list[Endpoint], *, rng: Any = None) -> None:
cloudra/benchmark/api_pool.py:80            super().__init__()
cloudra/benchmark/api_pool.py:81            self._endpoints = list(endpoints)
cloudra/benchmark/api_pool.py:82            # rng 可注入（测试用固定顺序）；生产默认 Random()
cloudra/benchmark/api_pool.py:83            self._rng = rng if rng is not None else random.Random()
cloudra/benchmark/api_pool.py:84
cloudra/benchmark/api_pool.py:85        def _make_client(self, settings):
cloudra/benchmark/api_pool.py:86            ep = _current_endpoint.get()
cloudra/benchmark/api_pool.py:87            if ep is None:
cloudra/benchmark/api_pool.py:88                return super()._make_client(settings)
cloudra/benchmark/api_pool.py:89            from cloudra.infra.llm_client import LLMClient
cloudra/benchmark/api_pool.py:90
cloudra/benchmark/api_pool.py:91            return LLMClient(
cloudra/benchmark/api_pool.py:92                api_key=ep.api_key or settings.api_key,
cloudra/benchmark/api_pool.py:93                base_url=ep.base_url or settings.base_url,
cloudra/benchmark/api_pool.py:94                timeout=settings.timeout,
cloudra/benchmark/api_pool.py:95            )
cloudra/benchmark/api_pool.py:96
cloudra/benchmark/api_pool.py:97        async def complete(self, agent, messages, tools, on_delta=None):
cloudra/benchmark/api_pool.py:98            if not self._endpoints:
cloudra/benchmark/api_pool.py:99                return await super().complete(agent, messages, tools, on_delta)
cloudra/benchmark/api_pool.py:100           # 每次调用随机洗牌端点顺序：既摊初始负载，又保证 429 时不重复试同一个
cloudra/benchmark/api_pool.py:101           order = self._rng.sample(self._endpoints, k=len(self._endpoints))
cloudra/benchmark/api_pool.py:102           last_exc: Optional[BaseException] = None
cloudra/benchmark/api_pool.py:103           for ep in order:
cloudra/benchmark/api_pool.py:104               token = _current_endpoint.set(ep)
cloudra/benchmark/api_pool.py:105               try:
cloudra/benchmark/api_pool.py:106                   return await super().complete(agent, messages, tools, on_delta)
cloudra/benchmark/api_pool.py:107               except BaseException as exc:
cloudra/benchmark/api_pool.py:108                   if not _is_rate_limit(exc):
cloudra/benchmark/api_pool.py:109                       raise
cloudra/benchmark/api_pool.py:110                   last_exc = exc
cloudra/benchmark/api_pool.py:111                   logger.warning("endpoint %s rate-limited (429); trying next", ep.
display())
cloudra/benchmark/api_pool.py:112               finally:
cloudra/benchmark/api_pool.py:113                   _current_endpoint.reset(token)
cloudra/benchmark/api_pool.py:114           assert last_exc is not None
cloudra/benchmark/api_pool.py:115           raise last_exc
cloudra/benchmark/backend.py:1  """runtime_v2-backed agent backend for the benchmark runner.
cloudra/benchmark/backend.py:2
cloudra/benchmark/backend.py:3  Per task this spins up an in-process runtime_v2 kernel (RuntimeCore +
```

```
cloudra/benchmark/backend.py:4  TurnPipeline + AgentRunner), spawns the task's coordinator agent with the task
cloudra/benchmark/backend.py:5  query against the per-task sandbox as the project workspace, waits for the
cloudra/benchmark/backend.py:6  task to genuinely finish (see ``watch.py`` for the wait_background /
cloudra/benchmark/backend.py:7  wait_review state machine, GH #30), then returns the agent's final message
cloudra/benchmark/backend.py:8  plus the barrier extracted from it (the final message is the input to refusal
cloudra/benchmark/backend.py:9  classification — see `refusal.py`). The method whitelist is enforced by
cloudra/benchmark/backend.py:10 appending `MethodPolicyStage` to the pipeline; compute jobs submitted via
cloudra/benchmark/backend.py:11 `submit_compute_job` are claimed and executed by a per-task in-process
cloudra/benchmark/backend.py:12 `SchedulerDaemon` (production runs it as a separate `cloudra scheduler`
cloudra/benchmark/backend.py:13 process instead).
cloudra/benchmark/backend.py:14
cloudra/benchmark/backend.py:15 Process-wide isolation is the caller's job: the benchmark CLI points the
cloudra/benchmark/backend.py:16 global config's `data_dir` at the benchmark data dir before any tool runs, so
cloudra/benchmark/backend.py:17 session stores, compute jobs and MEMORY.md never touch the live conversation
cloudra/benchmark/backend.py:18 data dir. LLM credentials still come from the normal global config slots
cloudra/benchmark/backend.py:19 (coordinator/follower), unchanged.
cloudra/benchmark/backend.py:20 """
cloudra/benchmark/backend.py:21
cloudra/benchmark/backend.py:22 from __future__ import annotations
cloudra/benchmark/backend.py:23
cloudra/benchmark/backend.py:24 import asyncio
cloudra/benchmark/backend.py:25 import logging
cloudra/benchmark/backend.py:26 import os
cloudra/benchmark/backend.py:27 import threading
cloudra/benchmark/backend.py:28 from pathlib import Path
cloudra/benchmark/backend.py:29 from typing import Any
cloudra/benchmark/backend.py:30
cloudra/benchmark/backend.py:31 from cloudra.benchmark.policy_stage import MethodPolicyStage
cloudra/benchmark/backend.py:32 from cloudra.benchmark.schema import TaskSpec
cloudra/benchmark/backend.py:33 from cloudra.benchmark.watch import RunWatcher
cloudra/benchmark/backend.py:34
cloudra/benchmark/backend.py:35 logger = logging.getLogger(__name__)
cloudra/benchmark/backend.py:36
cloudra/benchmark/backend.py:37 class RuntimeV2Backend:
cloudra/benchmark/backend.py:38     """`AgentBackend` implementation: run each task on the real runtime_v2.
cloudra/benchmark/backend.py:39
cloudra/benchmark/backend.py:40     `llm` / `tools` are injectable so tests can run the
cloudra/benchmark/backend.py:41     whole wiring hermetically; production defaults are the legacy bridges.
cloudra/benchmark/backend.py:42     """
cloudra/benchmark/backend.py:43
cloudra/benchmark/backend.py:44     def __init__(
cloudra/benchmark/backend.py:45         self,
cloudra/benchmark/backend.py:46         *,
cloudra/benchmark/backend.py:47         run_timeout_s: float = 7200.0,
cloudra/benchmark/backend.py:48         stuck_grace_s: float = 300.0,
cloudra/benchmark/backend.py:49         poll_interval_s: float = 2.0,
cloudra/benchmark/backend.py:50         llm: Any = None,
cloudra/benchmark/backend.py:51         tools: Any = None,
cloudra/benchmark/backend.py:52         start_scheduler: bool = True,
cloudra/benchmark/backend.py:53         store_path: Any = None,
cloudra/benchmark/backend.py:54     ) -> None:
cloudra/benchmark/backend.py:55         self.run_timeout_s = run_timeout_s
```

```
56 cloudra/benchmark/backend.py:56          self.stuck_grace_s = stuck_grace_s
57 cloudra/benchmark/backend.py:57          self.poll_interval_s = poll_interval_s
58 cloudra/benchmark/backend.py:58          self.llm = llm
59 cloudra/benchmark/backend.py:59          self.tools = tools
60 cloudra/benchmark/backend.py:60          self.start_scheduler = start_scheduler
61 cloudra/benchmark/backend.py:61          # 事件级持久化（可选但 CLI 默认开）：全战役共享一个 runtime-v2.db，
62 cloudra/benchmark/backend.py:62          # 只 append 不 restore——benchmark core 是一次性的，恢复语义无意义；
63 cloudra/benchmark/backend.py:63          # 崩溃后事件历史与 token 遥测（cloudra stats tokens）仍可查。
64 cloudra/benchmark/backend.py:64          self.store_path = Path(store_path) if store_path is not None else None
65 cloudra/benchmark/backend.py:65          # 每次任务的 RuntimeCore 按 task_id 留存，供测试与调试内省事件日志；
66 cloudra/benchmark/backend.py:66          # 并行跑题时多线程并发写，用锁保护。
67 cloudra/benchmark/backend.py:67          self._cores: dict[str, Any] = {}
68 cloudra/benchmark/backend.py:68          self._cores_lock = threading.Lock()
69 cloudra/benchmark/backend.py:69
70 cloudra/benchmark/backend.py:70      @property
71 cloudra/benchmark/backend.py:71      def last_core(self) -> Any:
72 cloudra/benchmark/backend.py:72          """最近一次任务的 core（串行调试便利）；并行时用 core_by_task()。"""
73 cloudra/benchmark/backend.py:73          with self._cores_lock:
74 cloudra/benchmark/backend.py:74              return next(reversed(self._cores.values())) if self._cores else None
75 cloudra/benchmark/backend.py:75
76 cloudra/benchmark/backend.py:76      def core_by_task(self, task_id: str) -> Any:
77 cloudra/benchmark/backend.py:77          with self._cores_lock:
78 cloudra/benchmark/backend.py:78              return self._cores.get(task_id)
79 cloudra/benchmark/backend.py:79
80 cloudra/benchmark/backend.py:80      def __call__(self, task: TaskSpec, sandbox: Path) -> None:
81 cloudra/benchmark/backend.py:81          asyncio.run(self._run_task(task, Path(sandbox)))
82 cloudra/benchmark/backend.py:82
83 cloudra/benchmark/backend.py:83      async def _run_task(self, task: TaskSpec, sandbox: Path) -> None:
84 cloudra/benchmark/backend.py:84          from cloudra.runtime_v2.adapters import (
85 cloudra/benchmark/backend.py:85              LegacyLLMProvider,
86 cloudra/benchmark/backend.py:86              LegacyPromptProvider,
87 cloudra/benchmark/backend.py:87              LegacyToolExecutor,
88 cloudra/benchmark/backend.py:88              legacy_default_runtime_settings,
89 cloudra/benchmark/backend.py:89          )
90 cloudra/benchmark/backend.py:90          from cloudra.runtime_v2.agent_runner import AgentRunner
91 cloudra/benchmark/backend.py:91          from cloudra.runtime_v2.commands import CommandBus
92 cloudra/benchmark/backend.py:92          from cloudra.runtime_v2.core import RuntimeCore
93 cloudra/benchmark/backend.py:93          from cloudra.runtime_v2.policies import default_stages
94 cloudra/benchmark/backend.py:94          from cloudra.runtime_v2.schemas import Command
95 cloudra/benchmark/backend.py:95          from cloudra.runtime_v2.turn_pipeline import TurnPipeline
96 cloudra/benchmark/backend.py:96
97 cloudra/benchmark/backend.py:97          sandbox.mkdir(parents=True, exist_ok=True)
98 cloudra/benchmark/backend.py:98          sandbox = sandbox.resolve()
99 cloudra/benchmark/backend.py:99
100 cloudra/benchmark/backend.py:100             core = RuntimeCore(strict_events=True)
101 cloudra/benchmark/backend.py:101             with self._cores_lock:
102 cloudra/benchmark/backend.py:102                 self._cores[task.task_id] = core
103 cloudra/benchmark/backend.py:103             if self.store_path is not None:
104 cloudra/benchmark/backend.py:104                 from cloudra.runtime_v2.persistence import RuntimeStore
105 cloudra/benchmark/backend.py:105
106 cloudra/benchmark/backend.py:106                 # 与 services/api_v2/app.py 同款接法：多题共享一个库文件，append
107 cloudra/benchmark/backend.py:107                 # 经 to_thread 卸载，WAL + busy_timeout 由 RuntimeStore 自带。
```

```
cloudra/benchmark/backend.py:108                core.store = RuntimeStore(self.store_path)
cloudra/benchmark/backend.py:109
cloudra/benchmark/backend.py:110                async def persist_event(envelope) -> None:
cloudra/benchmark/backend.py:111                    # llm.chunk deltas 是瞬时 hub 事件：无回放价值，落库会灌爆事件表。
cloudra/benchmark/backend.py:112                    if envelope.type == "llm.chunk":
cloudra/benchmark/backend.py:113                        return
cloudra/benchmark/backend.py:114                    await asyncio.to_thread(core.store.append_event, envelope)
cloudra/benchmark/backend.py:115
cloudra/benchmark/backend.py:116                core.event_hub.subscribe(persist_event)
cloudra/benchmark/backend.py:117            core.runtime_defaults = legacy_default_runtime_settings()
cloudra/benchmark/backend.py:118            stage = MethodPolicyStage(task.method_policy, base_dir=sandbox)
cloudra/benchmark/backend.py:119            stages = [*default_stages(core), stage]
cloudra/benchmark/backend.py:120            executor = self.tools or LegacyToolExecutor(core)
cloudra/benchmark/backend.py:121            from cloudra.runtime_v2.invocation import InvocationService
cloudra/benchmark/backend.py:122
cloudra/benchmark/backend.py:123            invocations = InvocationService(
cloudra/benchmark/backend.py:124                event_writer=core.event_writer, stages=stages, tools=executor, agents=core.
agents
cloudra/benchmark/backend.py:125            )
cloudra/benchmark/backend.py:126            pipeline = TurnPipeline(
cloudra/benchmark/backend.py:127                event_log=core.event_log,
cloudra/benchmark/backend.py:128                stages=stages,
cloudra/benchmark/backend.py:129                llm=self.llm or LegacyLLMProvider(),
cloudra/benchmark/backend.py:130                tools=executor,
cloudra/benchmark/backend.py:131                prompt_provider=LegacyPromptProvider(),
cloudra/benchmark/backend.py:132                event_hub=core.event_hub,
cloudra/benchmark/backend.py:133                notices=core.notices,
cloudra/benchmark/backend.py:134                invocations=invocations,
cloudra/benchmark/backend.py:135            )
cloudra/benchmark/backend.py:136            core.runner = AgentRunner(core, pipeline)
cloudra/benchmark/backend.py:137            scheduler = None
cloudra/benchmark/backend.py:138            if self.start_scheduler:
cloudra/benchmark/backend.py:139                from cloudra.scheduler.daemon import SchedulerDaemon
cloudra/benchmark/backend.py:140
cloudra/benchmark/backend.py:141                # worker_id 含 pid+task_id：并行跑题时同进程多个实例不能共享
cloudra/benchmark/backend.py:142                # 默认的 scheduler:<pid>，否则租约归属互相串。
cloudra/benchmark/backend.py:143                scheduler = SchedulerDaemon(worker_id=f"scheduler:{os.getpid()}:{task.
task_id}")
cloudra/benchmark/backend.py:144                await scheduler.start()
cloudra/benchmark/backend.py:145
cloudra/benchmark/backend.py:146            agent = None
cloudra/benchmark/backend.py:147            bus = CommandBus(core)
cloudra/benchmark/backend.py:148            try:
cloudra/benchmark/backend.py:149                project = (
cloudra/benchmark/backend.py:150                    await bus.execute(
cloudra/benchmark/backend.py:151                        Command(
cloudra/benchmark/backend.py:152                            type="project.create",
cloudra/benchmark/backend.py:153                            payload={"name": f"benchmark:{task.task_id}", "workspace_dir":
str(sandbox)},
cloudra/benchmark/backend.py:154                        )
cloudra/benchmark/backend.py:155                    )
cloudra/benchmark/backend.py:156                ).data
cloudra/benchmark/backend.py:157                agent = (
cloudra/benchmark/backend.py:158                    await bus.execute(
cloudra/benchmark/backend.py:159                        Command(
```

```
cloudra/benchmark/backend.py:160                        type="agent.spawn",
cloudra/benchmark/backend.py:161                        target={"project_id": project.id},
cloudra/benchmark/backend.py:162                        payload={
cloudra/benchmark/backend.py:163                            "role_id": task.prompt.role,
cloudra/benchmark/backend.py:164                            "query": task.prompt.query,
cloudra/benchmark/backend.py:165                            "settings": {
cloudra/benchmark/backend.py:166                                "orchestration_mode": task.prompt.orchestration_mode.
    value,
cloudra/benchmark/backend.py:167                                # 无人值守：破坏性工具审批走 yolo 自动通道
cloudra/benchmark/backend.py:168                                "yolo": True,
cloudra/benchmark/backend.py:169                                # 无人值守（GH #31）：隐藏并 fail-closed 拦截
cloudra/benchmark/backend.py:170                                # ask_user，绝不允许挂进 waiting_input
cloudra/benchmark/backend.py:171                                "unattended": True,
cloudra/benchmark/backend.py:172                                # Each benchmark case is isolated from global
cloudra/benchmark/backend.py:173                                # cross-task memory and AGENTS injection.
cloudra/benchmark/backend.py:174                                "clean_context": True,
cloudra/benchmark/backend.py:175                            },
cloudra/benchmark/backend.py:176                        },
cloudra/benchmark/backend.py:177                    )
cloudra/benchmark/backend.py:178                )
cloudra/benchmark/backend.py:179                ).data
cloudra/benchmark/backend.py:180            await self._wait_terminal(bus, agent)
cloudra/benchmark/backend.py:181            # 评分只读交付物（图 + 日志 + 结构化拒答声明）：后端不回传任何
cloudra/benchmark/backend.py:182            # 供评分使用的文本/数字，杜绝"拿模型自述给自己打分"。
cloudra/benchmark/backend.py:183        finally:
cloudra/benchmark/backend.py:184            if agent is not None and core.runner is not None:
cloudra/benchmark/backend.py:185                await core.runner.cancel(agent.id)
cloudra/benchmark/backend.py:186            # 契约性关机补快照：core 接了 store 时把会话上下文投影落盘。
cloudra/benchmark/backend.py:187            await bus.flush_snapshot()
cloudra/benchmark/backend.py:188            if scheduler is not None:
cloudra/benchmark/backend.py:189                await scheduler.close()
cloudra/benchmark/backend.py:190
cloudra/benchmark/backend.py:191    async def _wait_terminal(self, bus, agent) -> None:
cloudra/benchmark/backend.py:192        """等题目真正结束（watch.RunWatcher 的 wait_background/wait_review
cloudra/benchmark/backend.py:193        语义，GH #30），卡死/超时则取消该题。"""
cloudra/benchmark/backend.py:194        from cloudra.persistence.database import ComputeDB
cloudra/benchmark/backend.py:195        from cloudra.runtime_v2.schemas import Command
cloudra/benchmark/backend.py:196        from cloudra.services.compute_store import acount_pending_compute_jobs
cloudra/benchmark/backend.py:197
cloudra/benchmark/backend.py:198        core = bus.core
cloudra/benchmark/backend.py:199        compute_db = ComputeDB()
cloudra/benchmark/backend.py:200
cloudra/benchmark/backend.py:201        async def background_pending() -> bool:
cloudra/benchmark/backend.py:202            # compute.db 是计算作业的唯一权威；reminders 与子 agent 存活由
cloudra/benchmark/backend.py:203            # watcher 的 core 投影承担（子 agent 见 has_running_peers）。
cloudra/benchmark/backend.py:204            if bool(core.reminders.pending(agent.id)):
cloudra/benchmark/backend.py:205                return True
cloudra/benchmark/backend.py:206            return bool(
cloudra/benchmark/backend.py:207                await acount_pending_compute_jobs(compute_db, agent.project_id)
cloudra/benchmark/backend.py:208            )
cloudra/benchmark/backend.py:209
cloudra/benchmark/backend.py:210        watcher = RunWatcher(
cloudra/benchmark/backend.py:211            run_timeout_s=self.run_timeout_s,
```